(Software) Product Line Engineering

Camille SALINESI

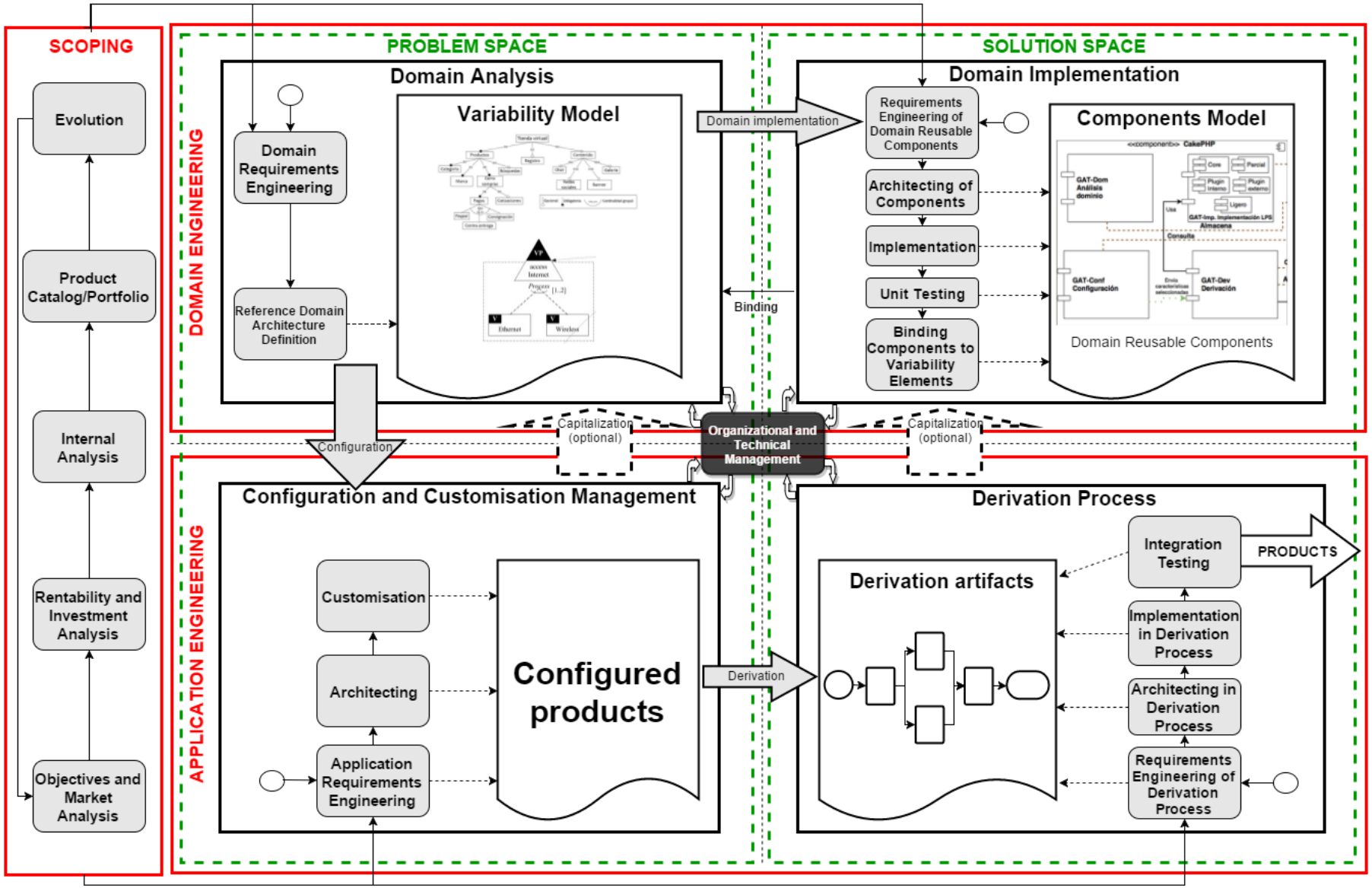
Camilo CORREA RESTREO

UNIVERSITÉ PARIS 1
PANTHÉON SORBONNE

Objectif and schedule of the course

Help you integrate Product Line Engineering (PLE):

- ✓in your developmen t practice &
- ✓in your research agenda



In this part:

Domain engineering = Engineering *for reuse*

1. Analysis
   A. Some languages to specify PLMs
      a. Feature Models
      b. OVM
      c. DOPLER
      d. Constraint-based product line models
   B. Strategies to specify the requirements of a PL
      a. Requirements-driven strategy
      b. Low-end vs. high-end product strategy (reference-product strategy)
      c. Multiviewpoint strategy
   C. Verification, diagnosis and fixing processes
2. Design
3. Implementation

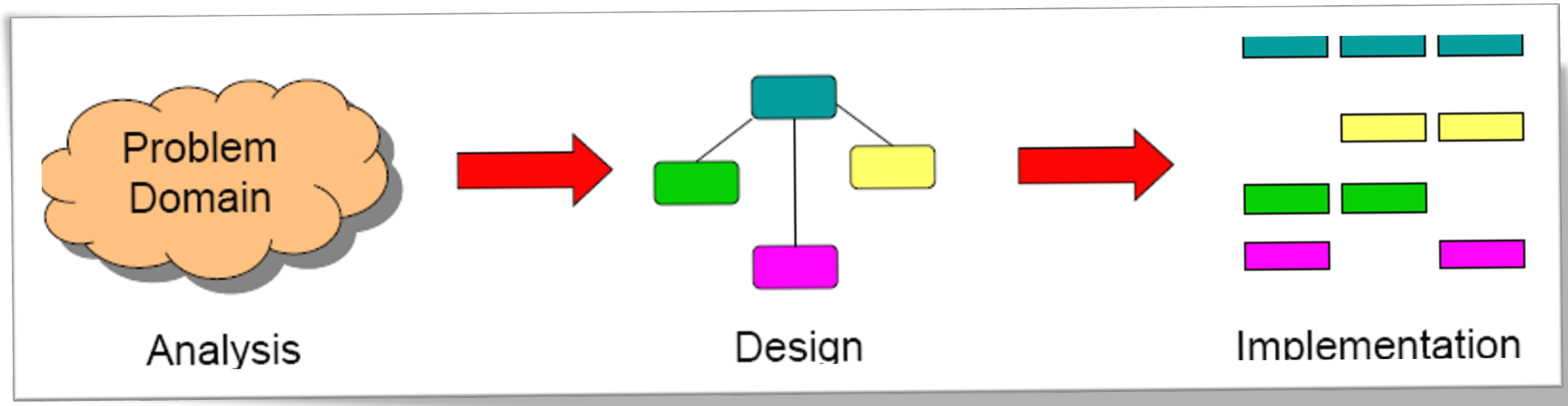


1. Analysis

A. Some languages to specify PLMs

a. Feature Models
b. OVM
c. DOPLER
d. Constraint-based product line models

Domain Engineering: it's all about the family



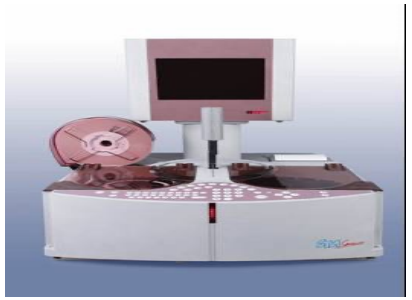
Three main activities:

1. **Analysis: Identify the variability present in the product line**
2. **Design:** Develop an architecture and production plan
3. **Implementation:** Develop all features with composability in mind

Extensive representation: Matrix representation







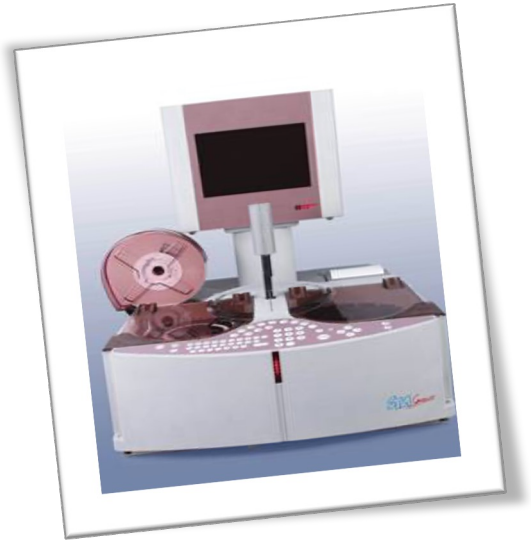
Components	Basic	Medium	Enriched
…			
Chronometric reaction measure	1	1	2
Photometric reaction measure	0	0	1
Barcode productID	0	1	1
RFID product ID	0	0	1
Manual product ID	1	1	1
External system PID	0	1	1

Extensive representation: breakdown structure



Basic	Medium	Advanced
1 Chronometric reaction measure	**1 Chronometric reaction measure**	**2 Chronometric reaction measure**
1 Manual product ID	1 Barcode productID	1 Photometric reaction measure
	1 Manual product ID	1 Barcode productID
	1 External system PID	2 RFID product ID
		1 Manual product ID
		1 External system PID





Benefits

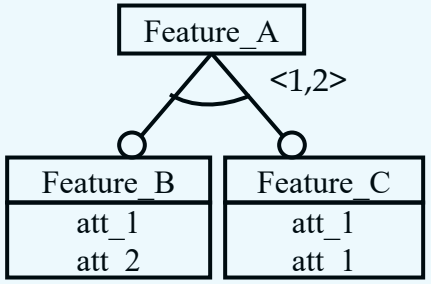
1. Big picture: clear vision of what is expected
2. Completeness: coverage of functional & physical variants
3. Consistency: avoid duplication of requirements by defining common & specific requirements
4. Easy to matin
   - At system level
   - At feature level

Scalability issue

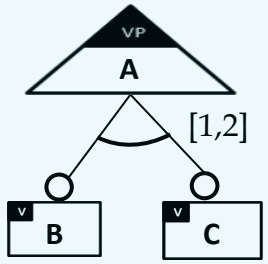


Product Line Models (PLMs): Intensive representation

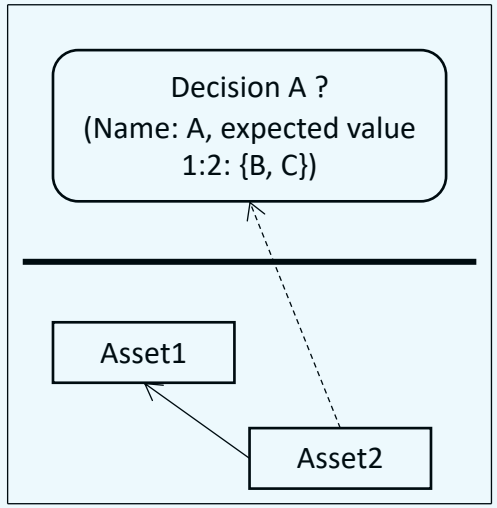
At the domain level



(Kang *et al.* 1990, Czarnecki *et al.* 2005, Schobbens *et al.* 2007, Michel *et al.* 2011)



(Pohl et al. 2005)



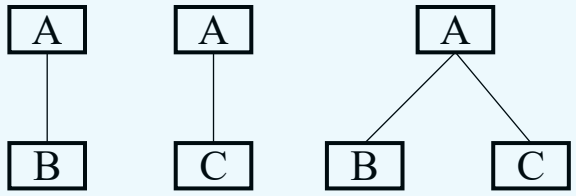
(Dhungana et al. 2006, 2010)

```
domain([A, B, C],
0, 1),
B ≤ A,
C ≤ A,
A ≤ B + C ≤ A * 2
```

(Salinesi et al. 2010, 2011)

Product Models (PMs): Extensive representation

At the product level

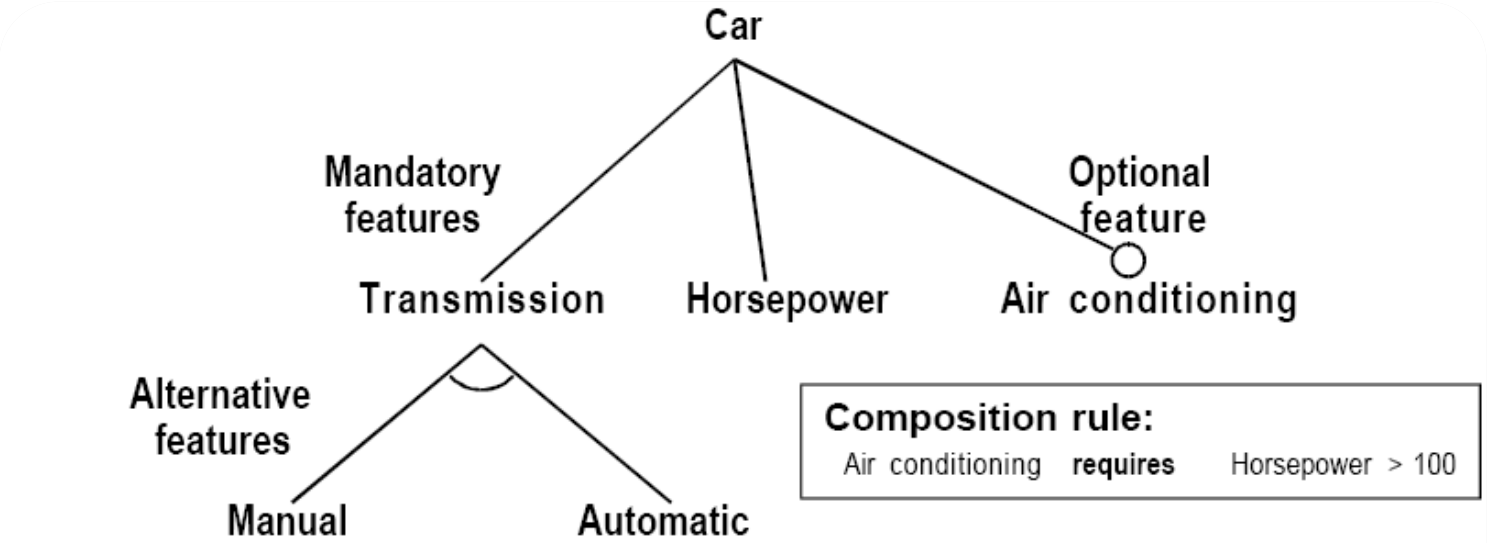



```
Decision A = true,
B = 4,
C = 9.8,
Asset1 = 1,
Asset2 = 1
```

```
P1 = {A, B, C}
P2 = {A, B}
P3 = {A, C}
```

Feature Models: First example of FODA (Feature-Oriented Domain Analysis)

Model:



Semantics:

C1: {Car, Transmission, Manual, Horsepower}
C2: {Car, Transmission, Automatic, Horsepower}
C3: {Car, Transmission, Manual, Horsepower > 100, Air conditioning}
C4: {Car, Transmission, Automatic, Horsepower > 100, Air conditioning}

Kang K., Cohen S., Hess J., Novak W., Peterson S. Feature-Oriented Domain Analysis (FODA) Feasibility Study, Technical Report CMU/SEI-90-TR-21, Software Engineering Institute, Carnegie Mellon University, November 1990.

A feature is…

- A *feature* is an essential **abstraction of a functionality or product characteristic** relevant to stakeholders (Czarnecki & Eisenecker, 2000)
- Features may be:
  - ✓abstract vs concrete
  - ✓common vs variable
- Feature VS {Classes, Functions, Aspects…} : external characteristics VS Internal details
- User-oriented needs VS developer, solution and quality concerns
- Intentions VS prescribed solutions

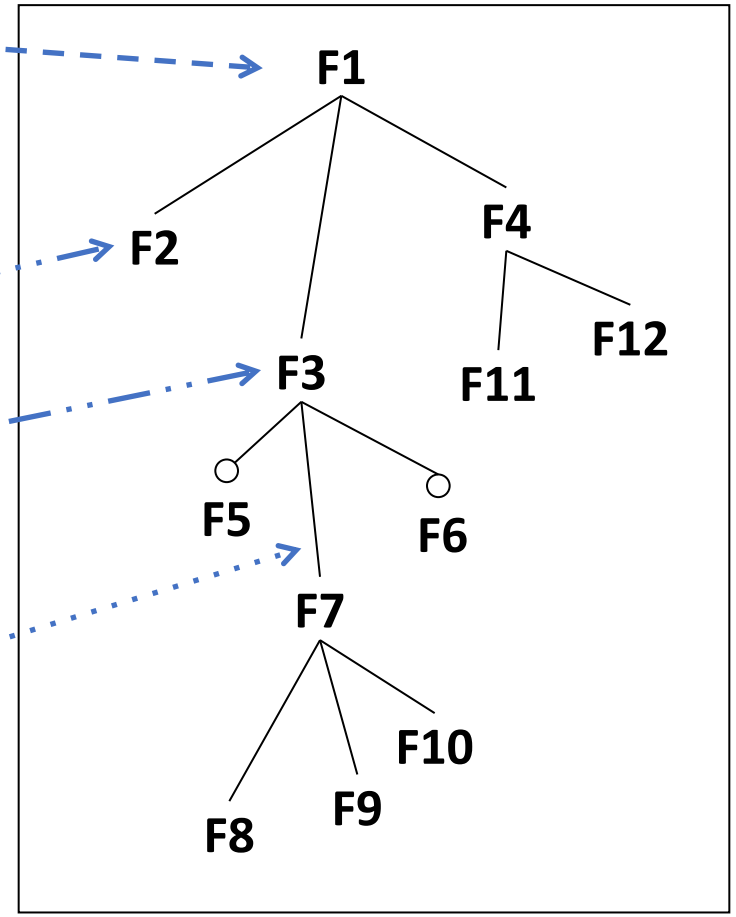


It can also be represented as a FODA diagram

Root: main concept of the PL

Nodes: features (concrete: F2, F5, F6, F8, F9, F10, F11, F12; abstract: F3, F4, F7)

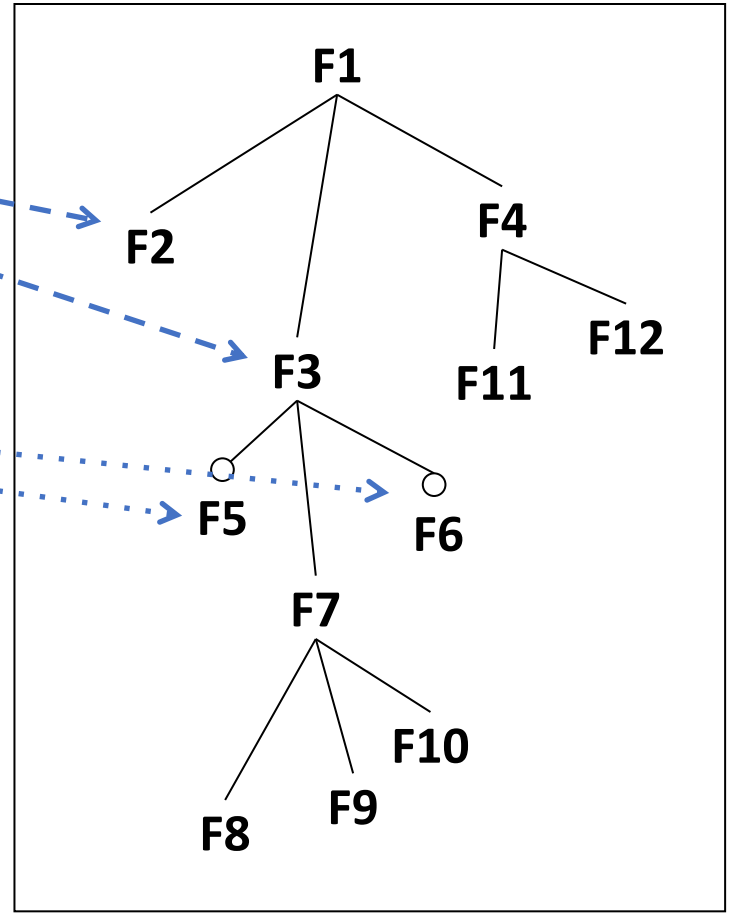
Tree: « composed of » relationships (refinement / decomposition)



FODA diagram

Mandatory features

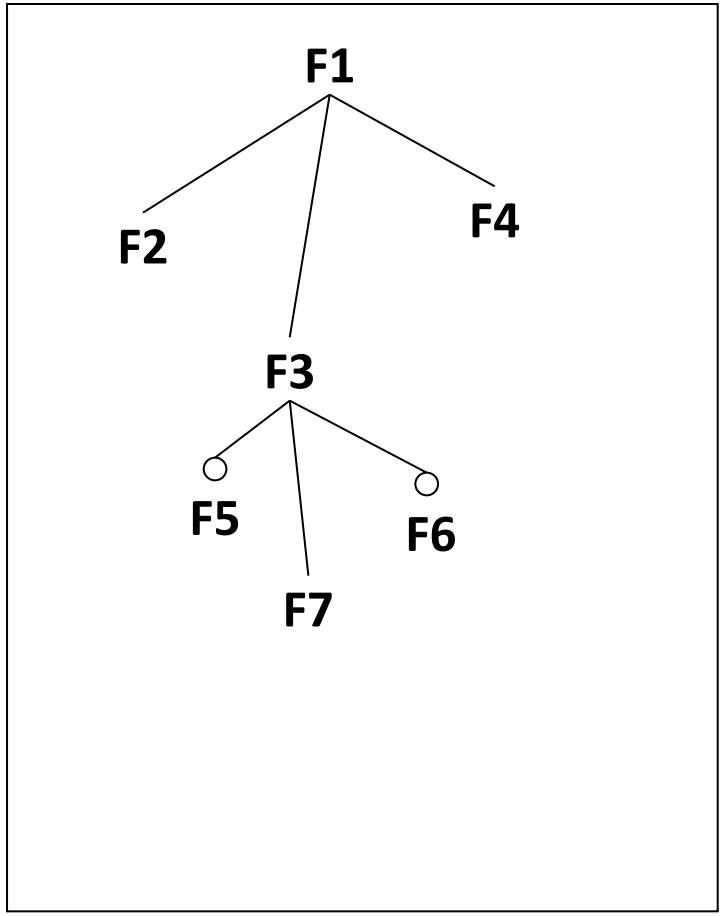
Optional features



FODA diagram

Semantics

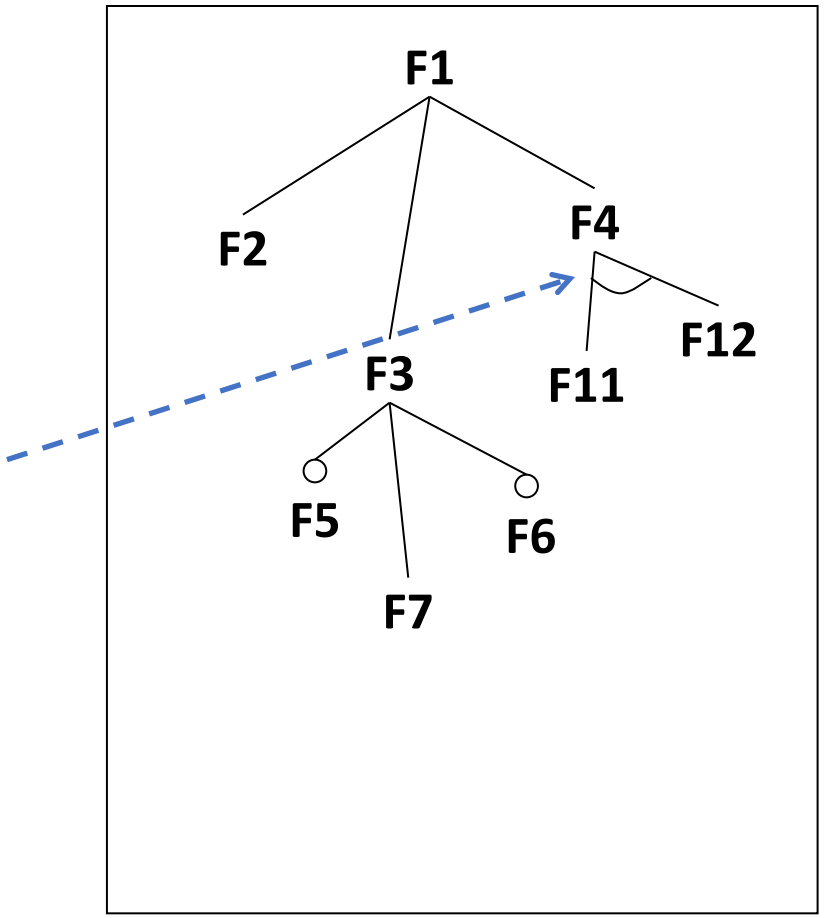
{
{F1, F2, F3, F4, F7},
{F1, F2, F3, F4, F5, F7},
{F1, F2, F3, F4, F6, F7},
{F1, F2, F3, F4, F5, F6, F7}
}



15

FODA diagram

- Mandatory features
- Optional features
- **Alternatives (XOR)**



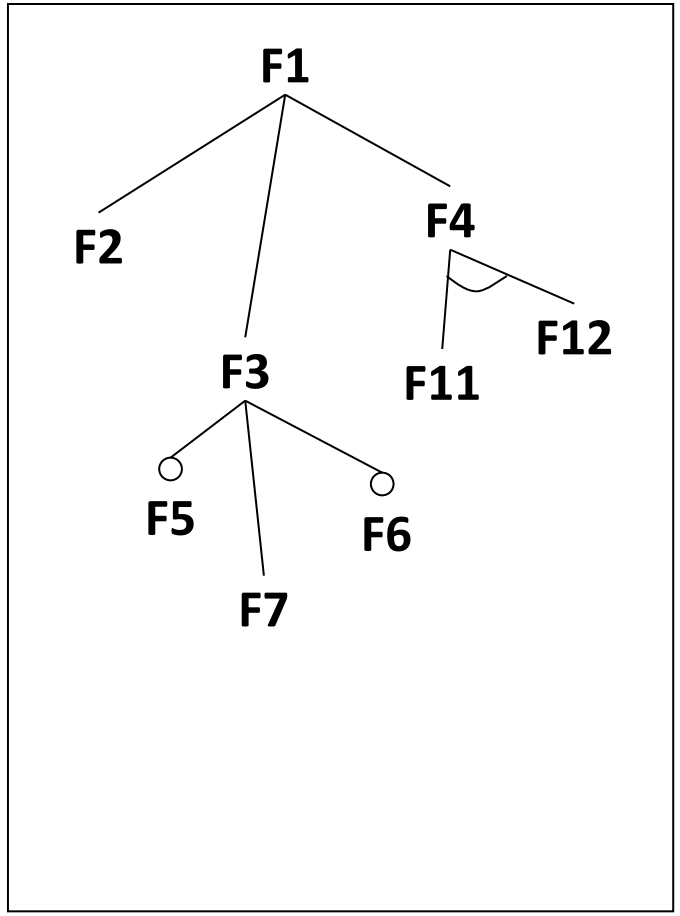
FODA diagram

Semantics

{

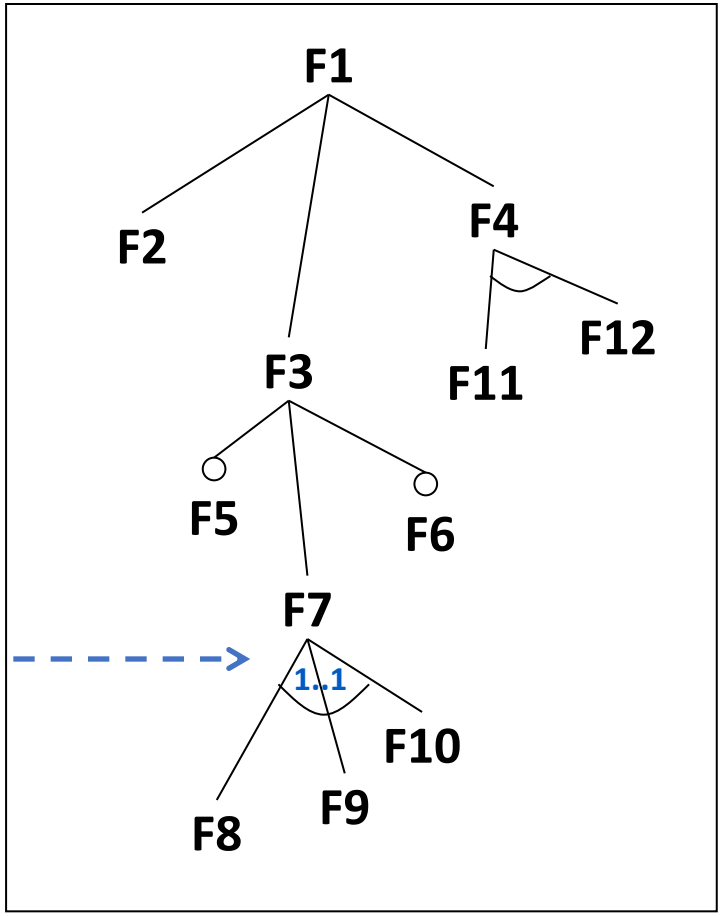
~~{F1, F2, F3, F4, F7},~~
~~{F1, F2, F3, F4, F5, F7},~~
~~{F1, F2, F3, F4, F6, F7},~~
~~{F1, F2, F3, F4, F5, F6, F7},~~ ⊘
{F1, F2, F3, F4, F7, F11},
{F1, F2, F3, F4, F5, F7, F11},
{F1, F2, F3, F4, F6, F7, F11},
{F1, F2, F3, F4, F5, F6, F7, F11}, F11
{F1, F2, F3, F4, F7, F12},
{F1, F2, F3, F4, F5, F7, F12},
{F1, F2, F3, F4, F6, F7, F12},
{F1, F2, F3, F4, F5, F6, F7, F12}, F12
~~{F1, F2, F3, F4, F7, F11, F12},~~
~~{F1, F2, F3, F4, F5, F7, F11, F12},~~
~~{F1, F2, F3, F4, F6, F7, F11, F12},~~
~~{F1, F2, F3, F4, F5, F6, F7, F11, F12}~~ F11 & F12

}



FODA diagram

- Mandatory features
- Optional features
- Alternatives (XOR)
- **Group cardinalities**



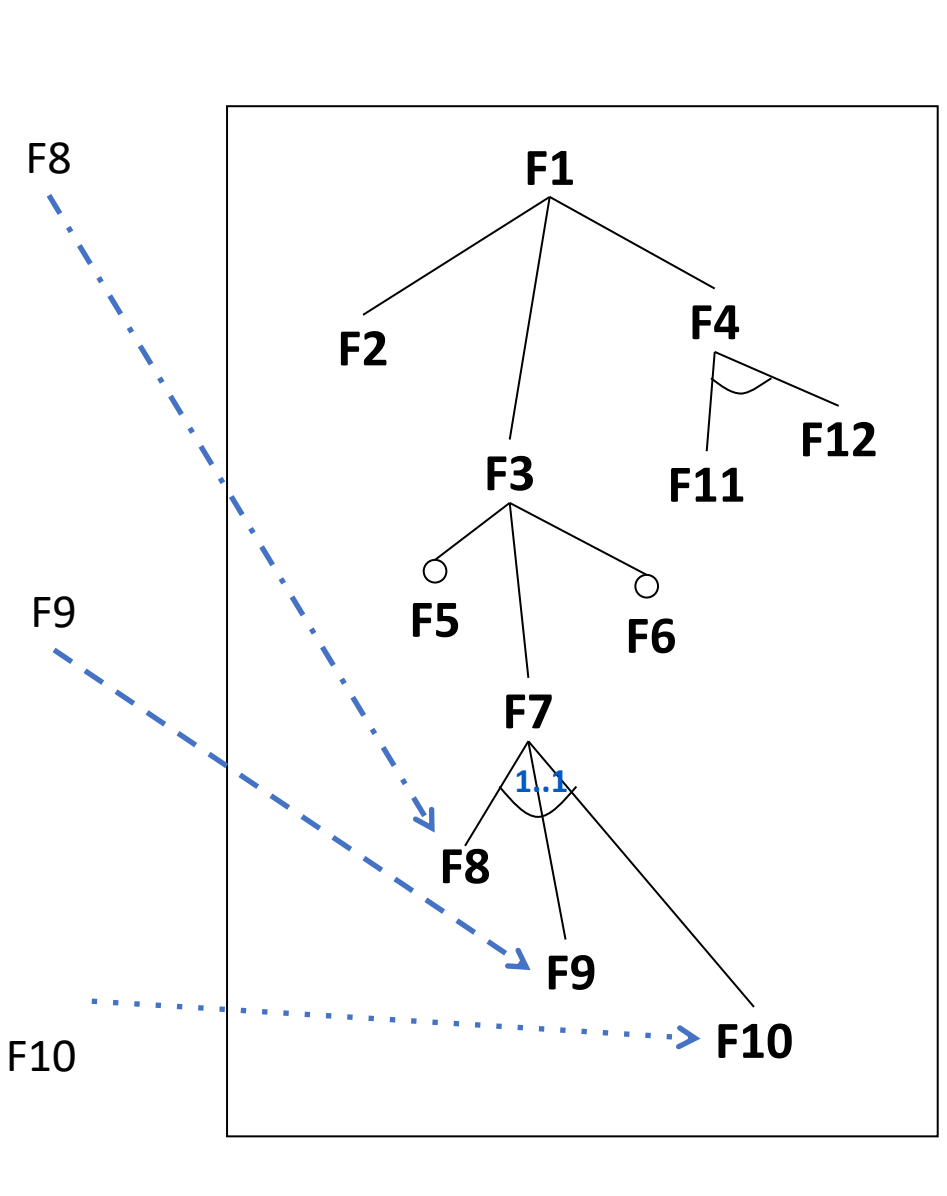
FODA diagram

Semantics
{

[F1,F2,F3,F4,F7,F8,F12],
[F1,F2,F3,F4,F7,F8,F11],
[F1,F2,F3,F4,F6,F7,F8,F12],
[F1,F2,F3,F4,F6,F7,F8,F11],
[F1,F2,F3,F4,F5,F7,F8,F12],
[F1,F2,F3,F4,F5,F7,F8,F11],
[F1,F2,F3,F4,F5,F6,F7,F8,F12],
[F1,F2,F3,F4,F5,F6,F7,F8,F11],

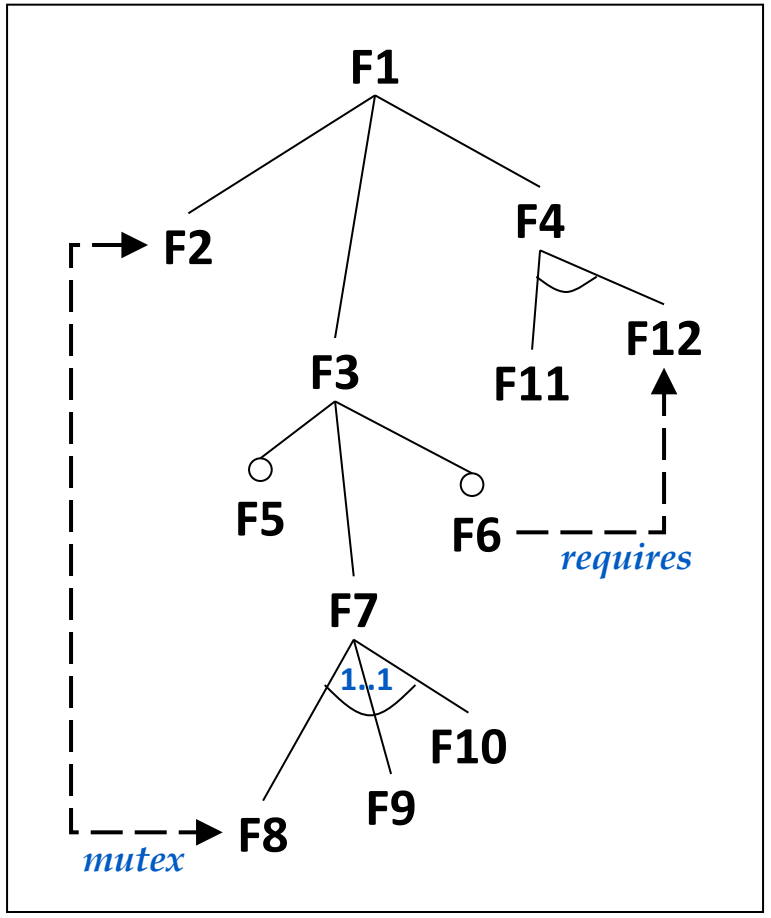
[F1,F2,F3,F4,F7,F9,F12],
[F1,F2,F3,F4,F7,F9,F11],
[F1,F2,F3,F4,F6,F7,F9,F12],
[F1,F2,F3,F4,F6,F7,F9,F11],
[F1,F2,F3,F4,F5,F7,F9,F12],
[F1,F2,F3,F4,F5,F7,F9,F11],
[F1,F2,F3,F4,F5,F6,F7,F9,F12],
[F1,F2,F3,F4,F5,F6,F7,F9,F11],

[F1,F2,F3,F4,F7,F10,F12],
[F1,F2,F3,F4,F7,F10,F12],
[F1,F2,F3,F4,F6,F7,F10,F12],
[F1,F2,F3,F4,F6,F7,F10,F11],
[F1,F2,F3,F4,F5,F7,F10,F12],
[F1,F2,F3,F4,F5,F7,F10,F11],
[F1,F2,F3,F4,F5,F6,F7,F10,F12],
[F1,F2,F3,F4,F5,F6,F7,F10,F11]
}



FODA diagram

- Mandatory features
- Optional features
- Alternatives (XOR)
- Group cardinalities
- **Dependencies**
  - **Requires**
  - **Mutex**



FODA diagram

Semantics

{

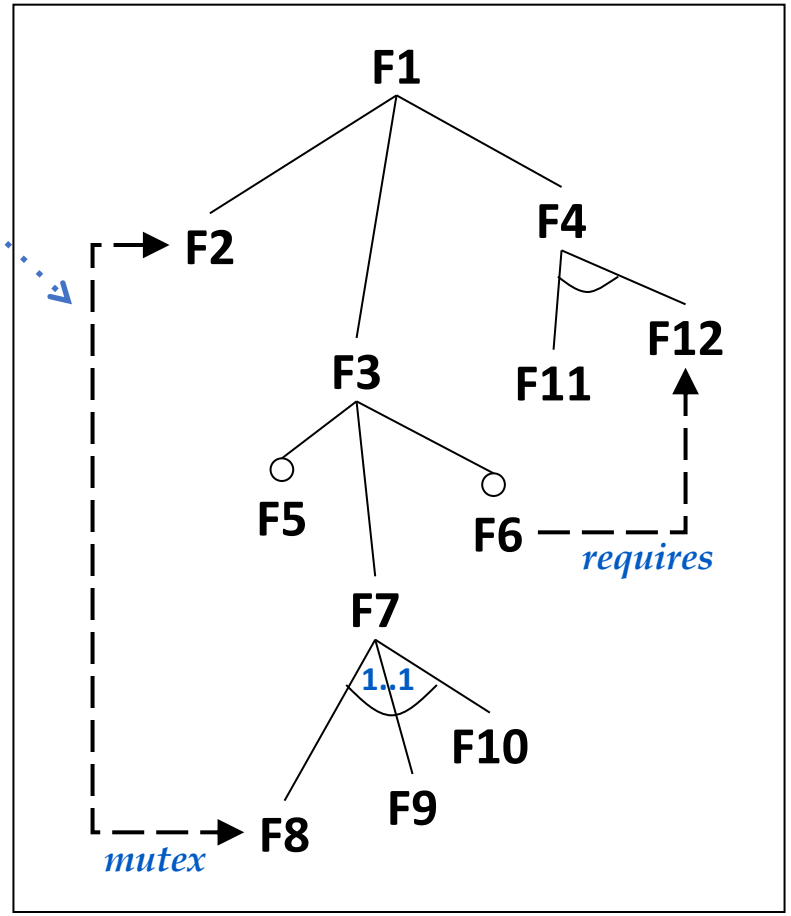
~~[F1,F2,F3,F4,F7,F8,F12],~~
~~[F1,F2,F3,F4,F7,F8,F11],~~
~~[F1,F2,F3,F4,F6,F7,F8,F12],~~
~~[F1,F2,F3,F4,F6,F7,F8,F11],~~
~~[F1,F2,F3,F4,F5,F7,F8,F12],~~
~~[F1,F2,F3,F4,F5,F7,F8,F11],~~
~~[F1,F2,F3,F4,F5,F6,F7,F8,F12],~~
~~[F1,F2,F3,F4,F5,F6,F7,F8,F11],~~

mutex

[F1,F2,F3,F4,F7,F9,F12],
[F1,F2,F3,F4,F7,F9,F11],
[F1,F2,F3,F4,F6,F7,F9,F12],
[F1,F2,F3,F4,F6,F7,F9,F11],
[F1,F2,F3,F4,F5,F7,F9,F12],
[F1,F2,F3,F4,F5,F7,F9,F11],
[F1,F2,F3,F4,F5,F6,F7,F9,F12],
[F1,F2,F3,F4,F5,F6,F7,F9,F11],

[F1,F2,F3,F4,F7,F10,F12],
[F1,F2,F3,F4,F7,F10,F12],
[F1,F2,F3,F4,F6,F7,F10,F12],
[F1,F2,F3,F4,F6,F7,F10,F11],
[F1,F2,F3,F4,F5,F7,F10,F12],
[F1,F2,F3,F4,F5,F7,F10,F11],
[F1,F2,F3,F4,F5,F6,F7,F10,F12],
[F1,F2,F3,F4,F5,F6,F7,F10,F11]

}



FODA diagram

Semantics

{

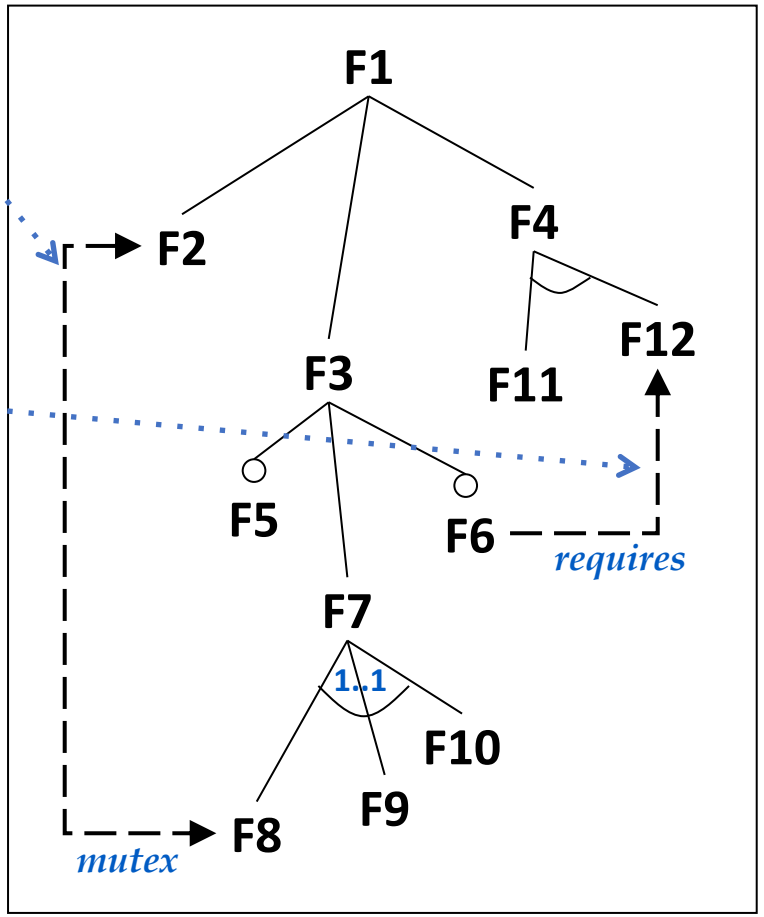
~~[F1,F2,F3,F4,F7,F8,F12],~~
~~[F1,F2,F3,F4,F7,F8,F11],~~
~~[F1,F2,F3,F4,F6,F7,F8,F12],~~
~~[F1,F2,F3,F4,F6,F7,F8,F11],~~
~~[F1,F2,F3,F4,F5,F7,F8,F12],~~
~~[F1,F2,F3,F4,F5,F7,F8,F11],~~
~~[F1,F2,F3,F4,F5,F6,F7,F8,F12],~~
~~[F1,F2,F3,F4,F5,F6,F7,F8,F11],~~

mutex

[F1,F2,F3,F4,F7,F9,F12],
[F1,F2,F3,F4,F7,F9,F11],
[F1,F2,F3,F4,F6,F7,F9,F12],
~~[F1,F2,F3,F4,F6,F7,F9,F11],~~
[F1,F2,F3,F4,F5,F7,F9,F12],
[F1,F2,F3,F4,F5,F7,F9,F11],
[F1,F2,F3,F4,F5,F6,F7,F9,F12],
~~[F1,F2,F3,F4,F5,F6,F7,F9,F11],~~

Requires

[F1,F2,F3,F4,F7,F10,F12],
[F1,F2,F3,F4,F7,F10,F12],
[F1,F2,F3,F4,F6,F7,F10,F12],
~~[F1,F2,F3,F4,F6,F7,F10,F11],~~
[F1,F2,F3,F4,F5,F7,F10,F12],
[F1,F2,F3,F4,F5,F7,F10,F11],
[F1,F2,F3,F4,F5,F6,F7,F10,F12],
~~[F1,F2,F3,F4,F5,F6,F7,F10,F11]~~
}

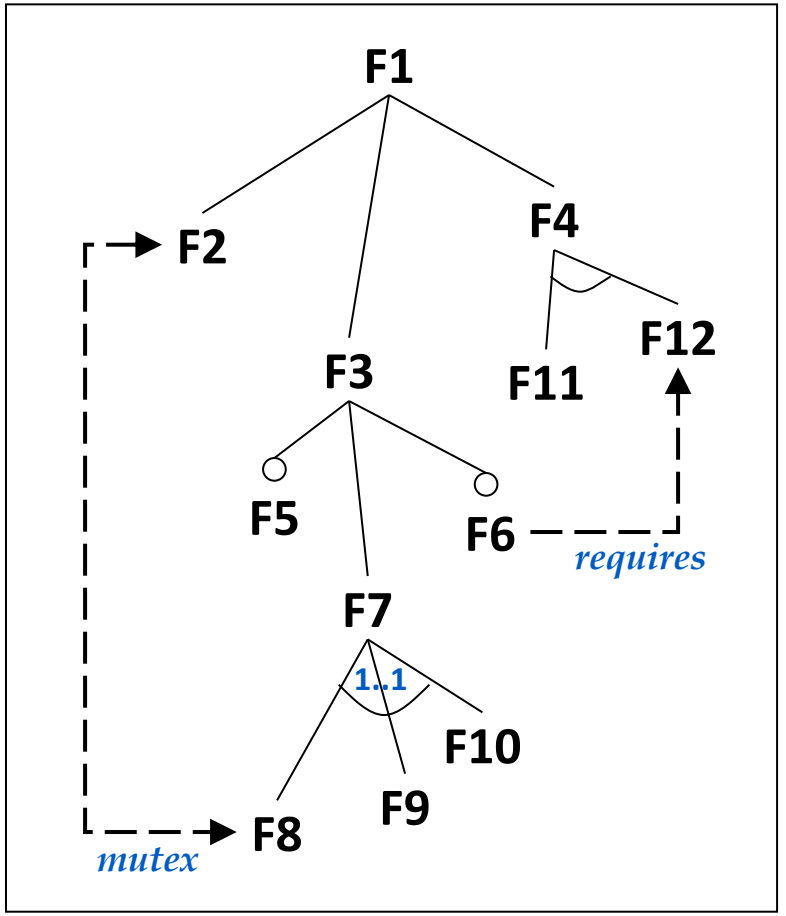


FODA diagram

Semantics

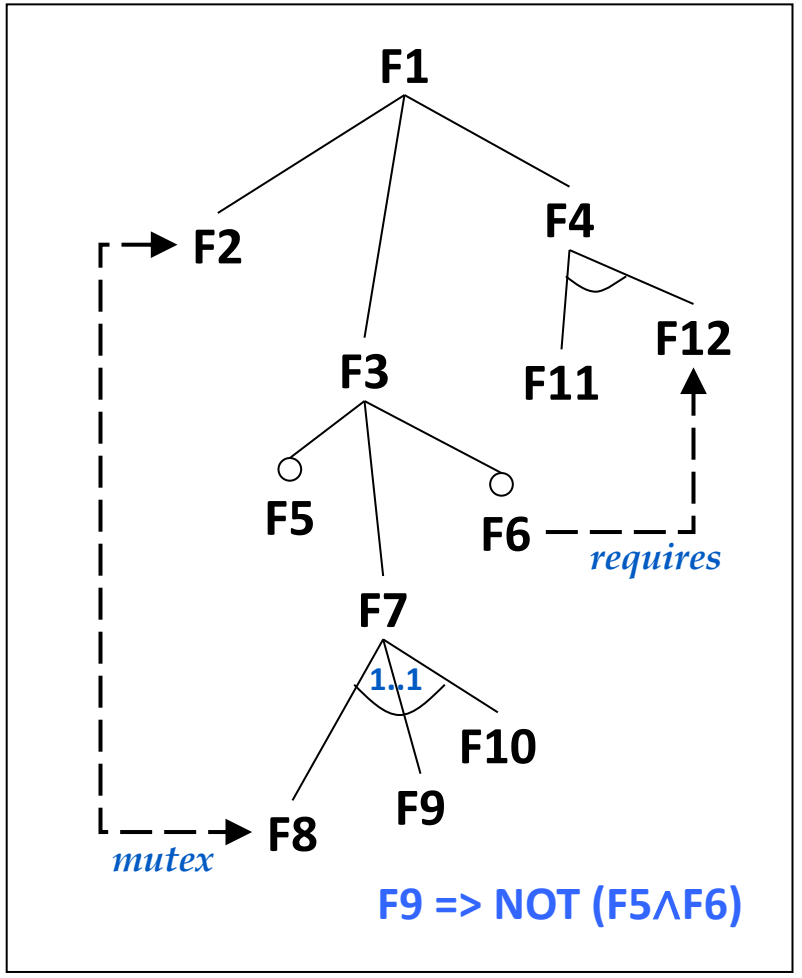
{

[F1,F2,F3,F4,F7,F9,F12],
[F1,F2,F3,F4,F7,F9,F11],
[F1,F2,F3,F4,F6,F7,F9,F12]
,
[F1,F2,F3,F4,F5,F7,F9,F12]
,
[F1,F2,F3,F4,F5,F7,F9,F11]
,
[F1,F2,F3,F4,F5,F6,F7,F9,F12],
[F1,F2,F3,F4,F7,F10,F12],
[F1,F2,F3,F4,F7,F10,F12],
[F1,F2,F3,F4,F6,F7,F10,F12],



FODA model

- Mandatory features
- Optional features
- Alternatives (XOR)
- Group cardinalities
- Dependencies
  - Requires
  - Mutex
- **Constraints**



FODA model

Semantics

{

[F1,F2,F3,F4,F7,F9,F12],
[F1,F2,F3,F4,F7,F9,F11],
[F1,F2,F3,F4,F6,F7,F9,F12]

,

[F1,F2,F3,F4,F5,F7,F9,F12]

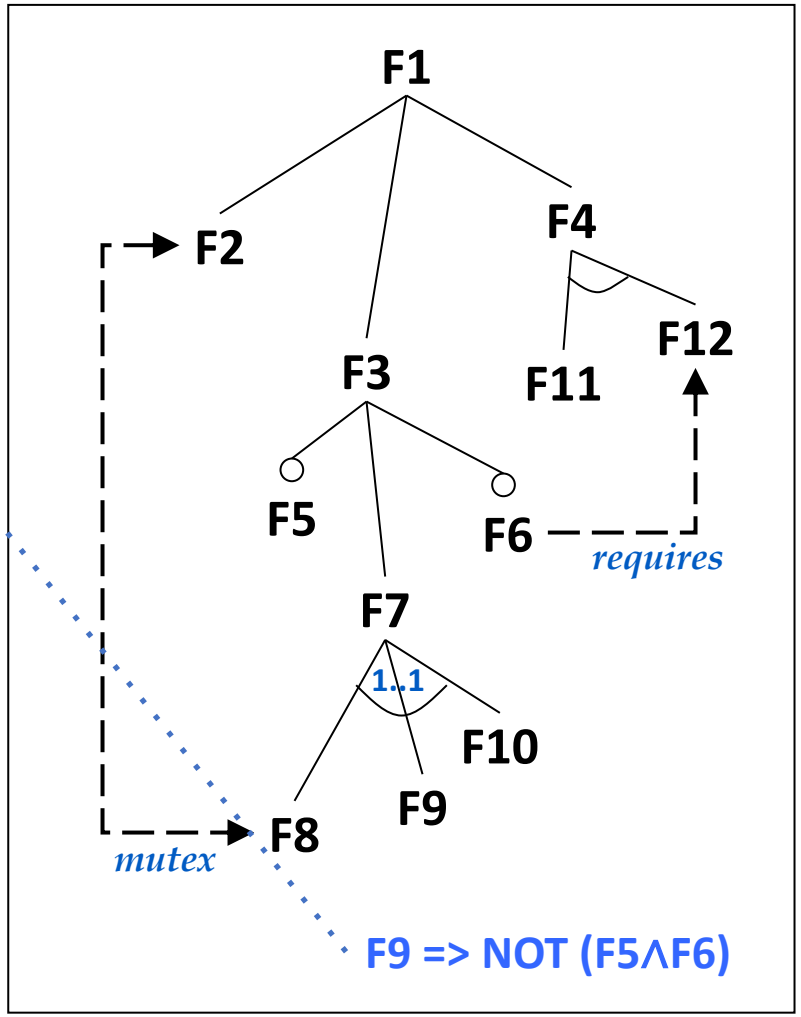
,

[F1,F2,F3,F4,F5,F7,F9,F11]

,

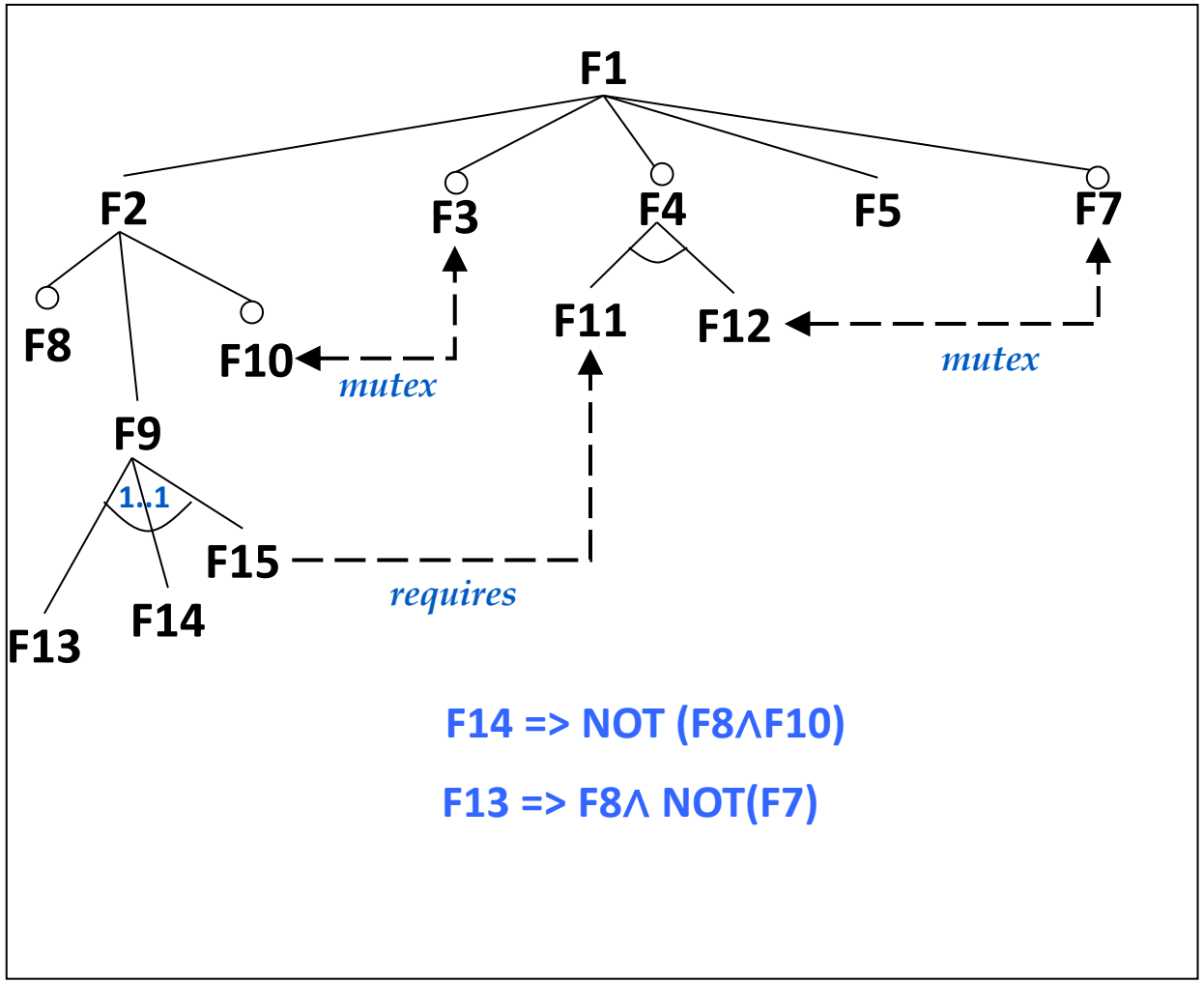
~~[F1,F2,F3,F4,F5,F6,F7,F9,F12],~~

[F1,F2,F3,F4,F7,F10,F12],
[F1,F2,F3,F4,F7,F10,F12],
[F1,F2,F3,F4,F6,F7,F10,F12
],



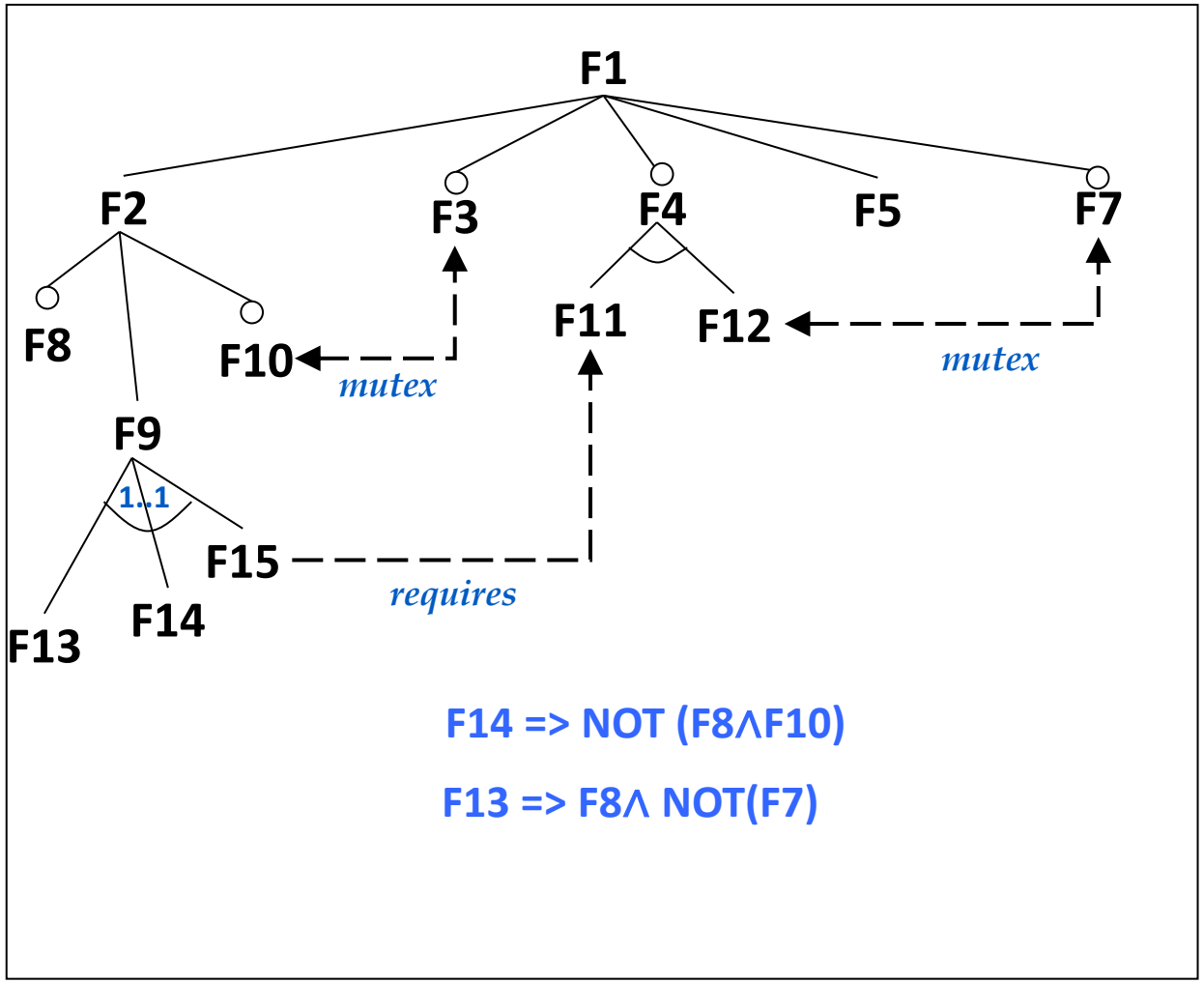
Exercise: Manual Analysis of FODA model

1. Construct at least 12 unique products from this model
2. What is the space of possible combinations with these features?
3. How many are actually allowed by the model?



Exercise: Manual Analysis of FODA model

- What is the space of possible combinations with these features?
- Answer: 2^13 combinations
- How many are actually allowed by the model?
- Answer: 46
- Why?



Feature modelling dialects: FORE

- **Optional** (F2 is optional regarding F1)



- **Mandatory** (F2 is madatory regarding F1)



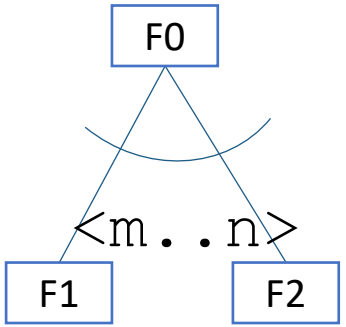
- **Requires** (F2 is required by F1)



- **Excludes** (F2 is excluded by F1, and viseversa)

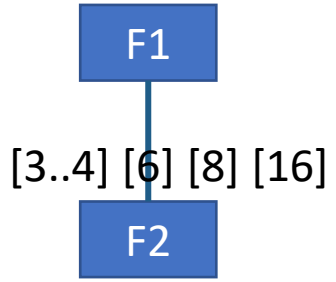


- **Group cardinality** (between `m` and `n` leaf features should be selected in a product if F0 is selected in the same product, and viceversa)

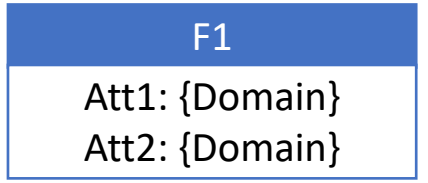


Concepts included into the FODA dialect

- **Individual Cardinalities**: Number of times a feature can occur in a configuration



- **Attributes**: specific measurable characteristics of a feature, with a name, a domain and a value at a given configuration time



- **Complex constraints**: Boolean, Arithmetic, Symbolic

all_different(…)

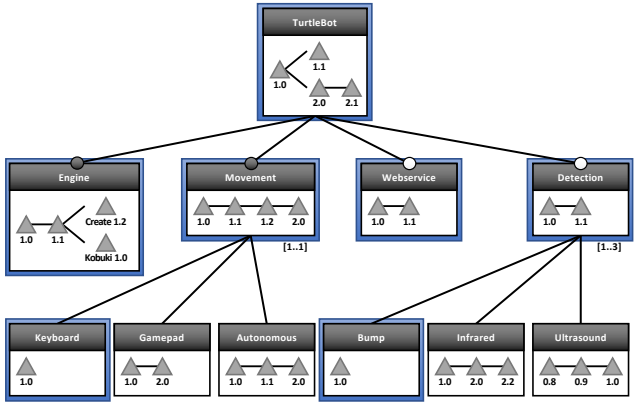
relation (…)

Equality

Inequality...

Cardinalities (Riebisch *et al. 2002, Czarnecki et al. 2005),*
Attributes (Streitferdt *et al. 2003, Benavides et al. 2005c, White et al. 2009).*
*Complex contraints (*Benavides *et al. 2005, Batory 2005, Salinesi et al. 2011)*

Evolution in (Schobbens *et al. 2007)*



+

encode

Version-Aware Constraints

1. Autonomous → Detection
2. Keyboard ∨ Gamepad → Webservice
3. Infrared [≥ 2.0] ∨ Ultrasound → Detection [≥ 1.1]
4. TurtleBot [≥ 2.0] → Engine [≥ Kobuki 1.0]
5. TurtleBot [1.0 – 1.1] → Engine [≤ Create 1.2]
6. TurtleBot [≥ 2.0] → ?Webservice [≥ 1.1]

Constraint Satisfaction Problem (CSP) Solver

solve

All Possible (Version) Configurations

select

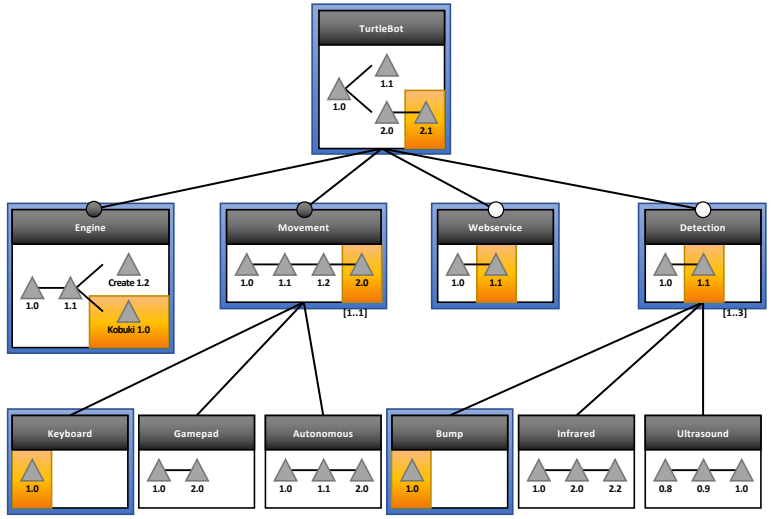
Scoring to find the "best" configuration

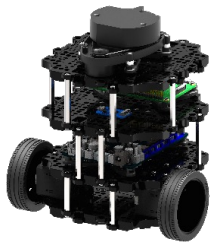
Requirement 1
Support for feature versions as units of configuration.

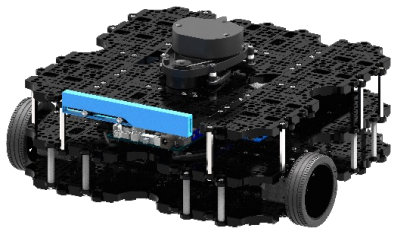
Requirement 2
Specification of the relation of versions in logical development lines (i.e., predecessor and successor versions including branching).

Requirement 3
Expression of dependencies on and incompatibilities with version ranges.

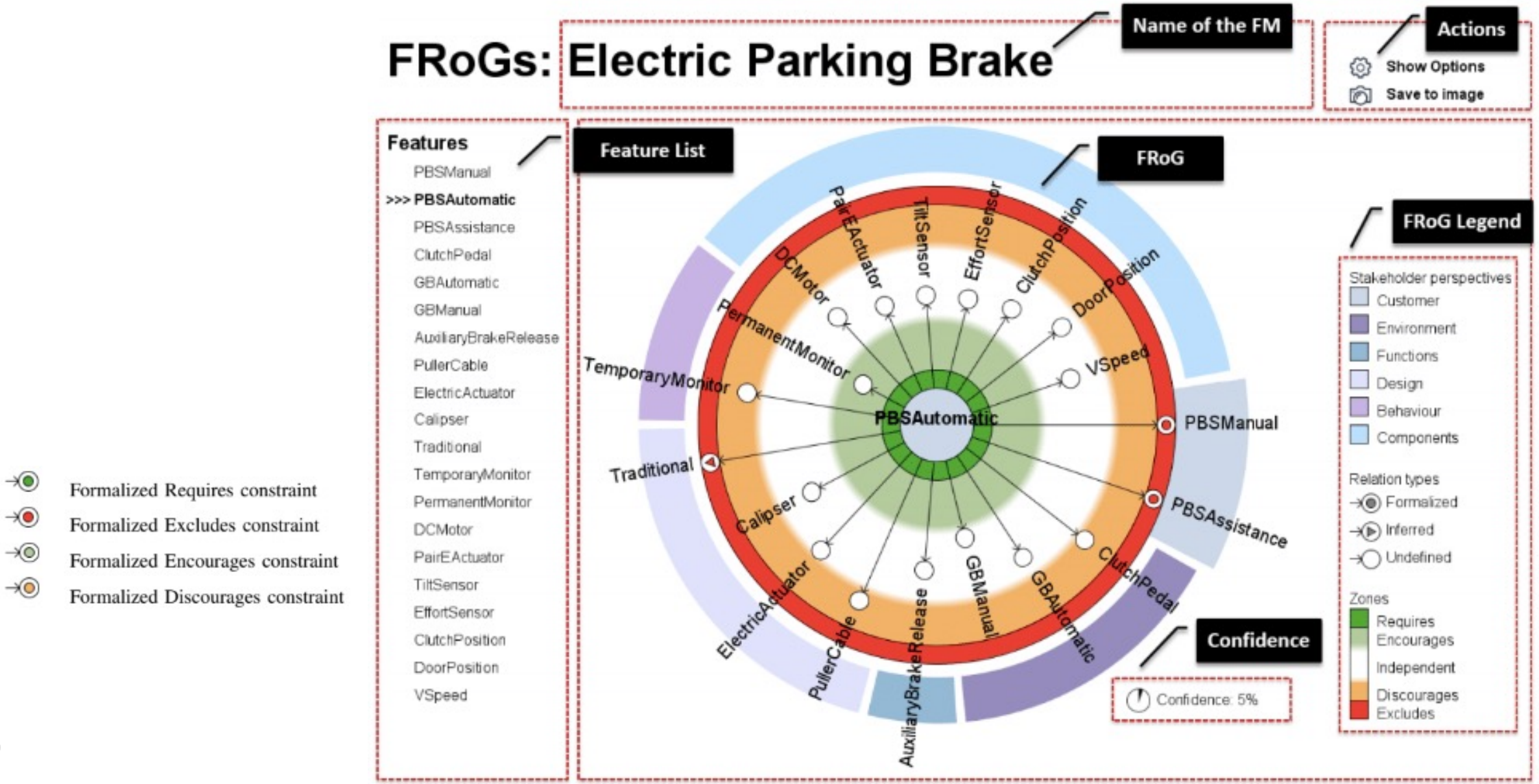
Requirement 4
Support for users in selecting suitable combinations of versions from a selection of features.



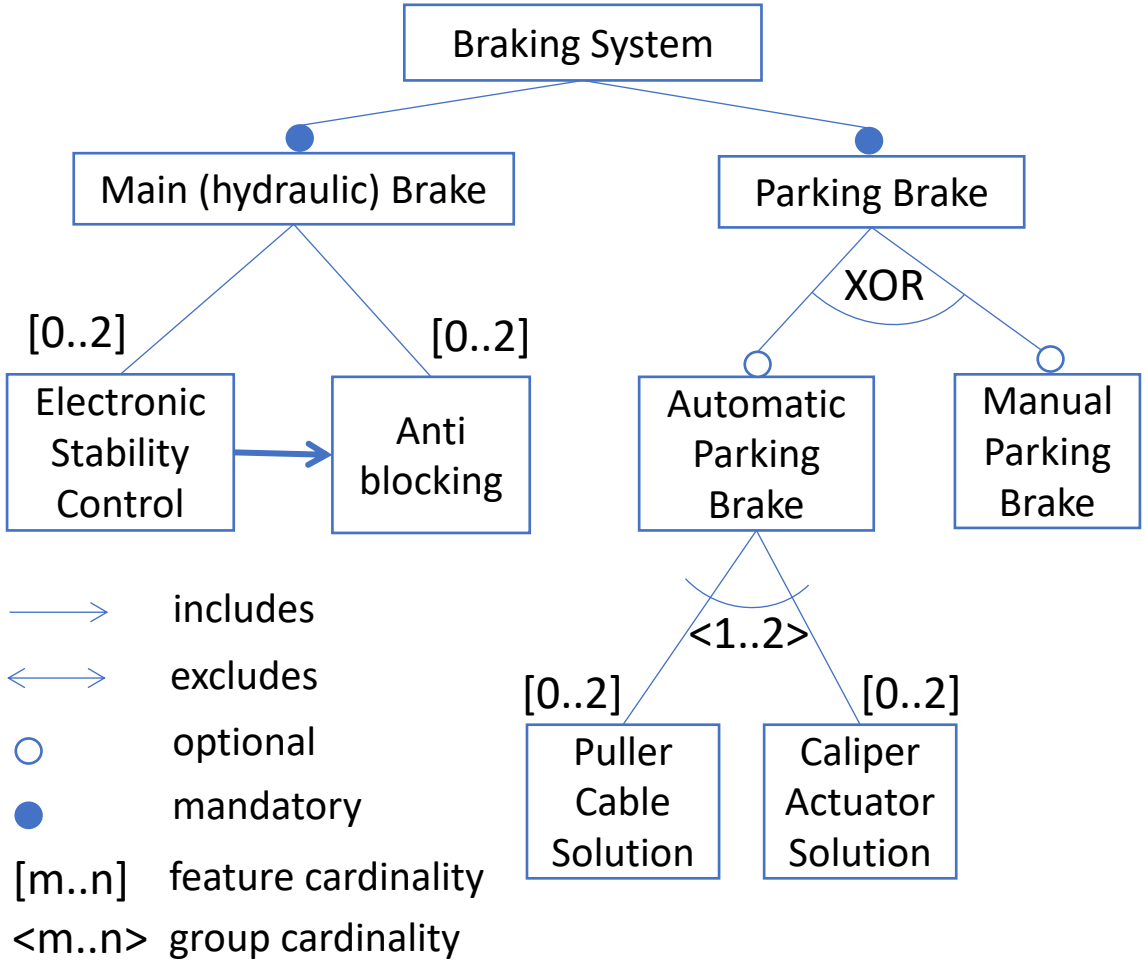




FRoG: Feature Relations Graphs (A Visualisation Paradigm for Feature Constraints in PLs)

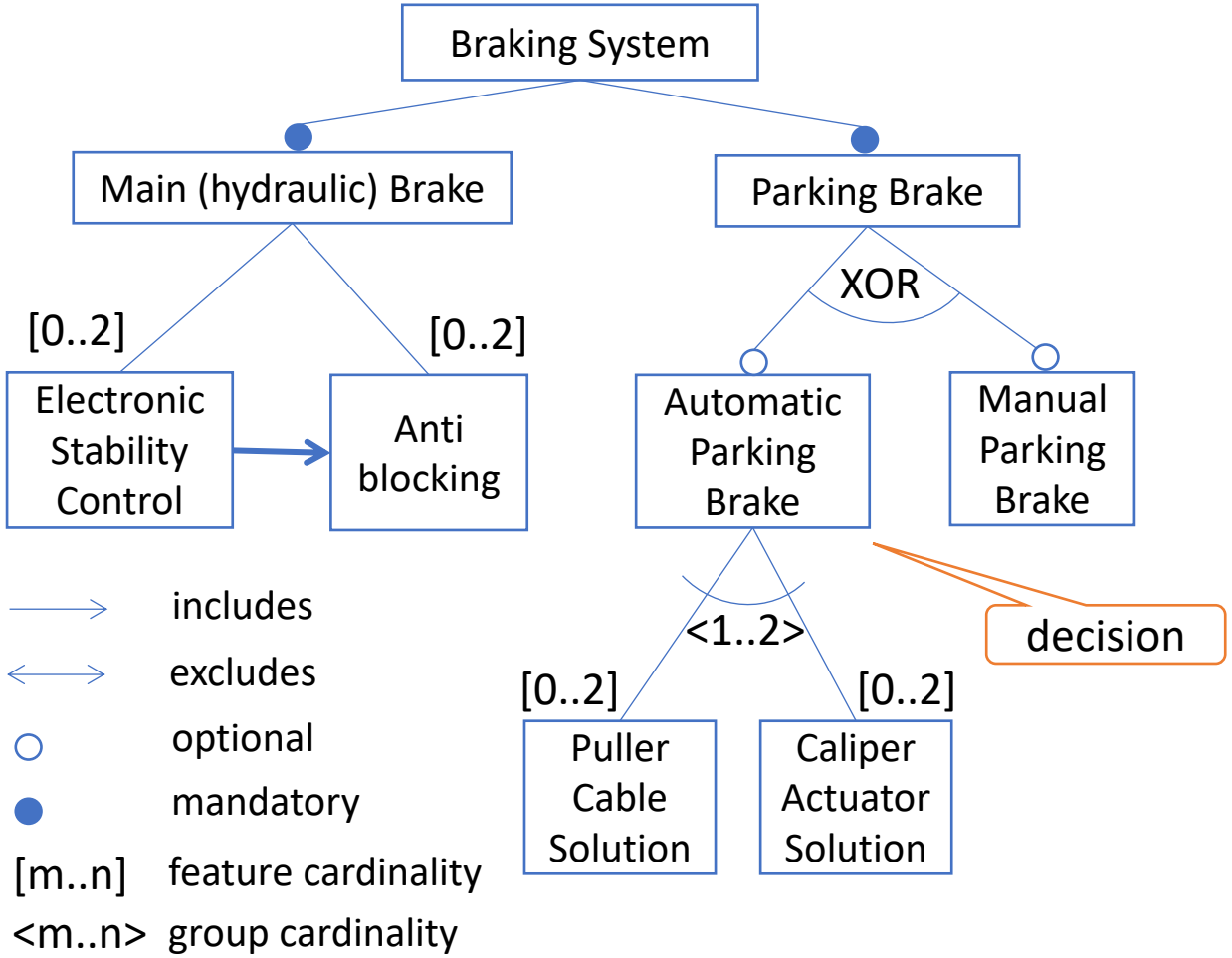


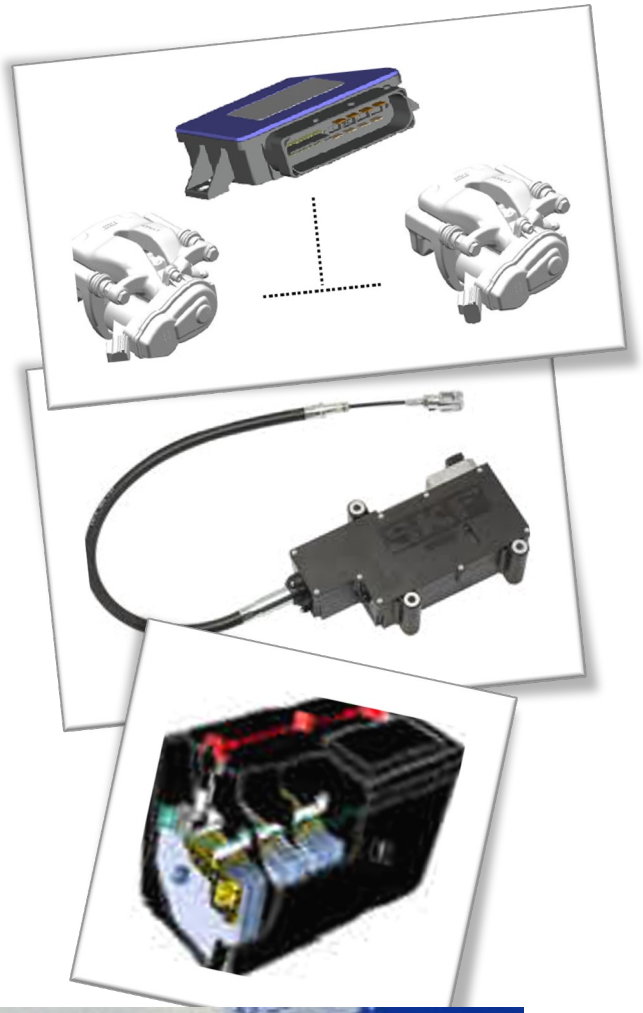
Some problems of the FMs





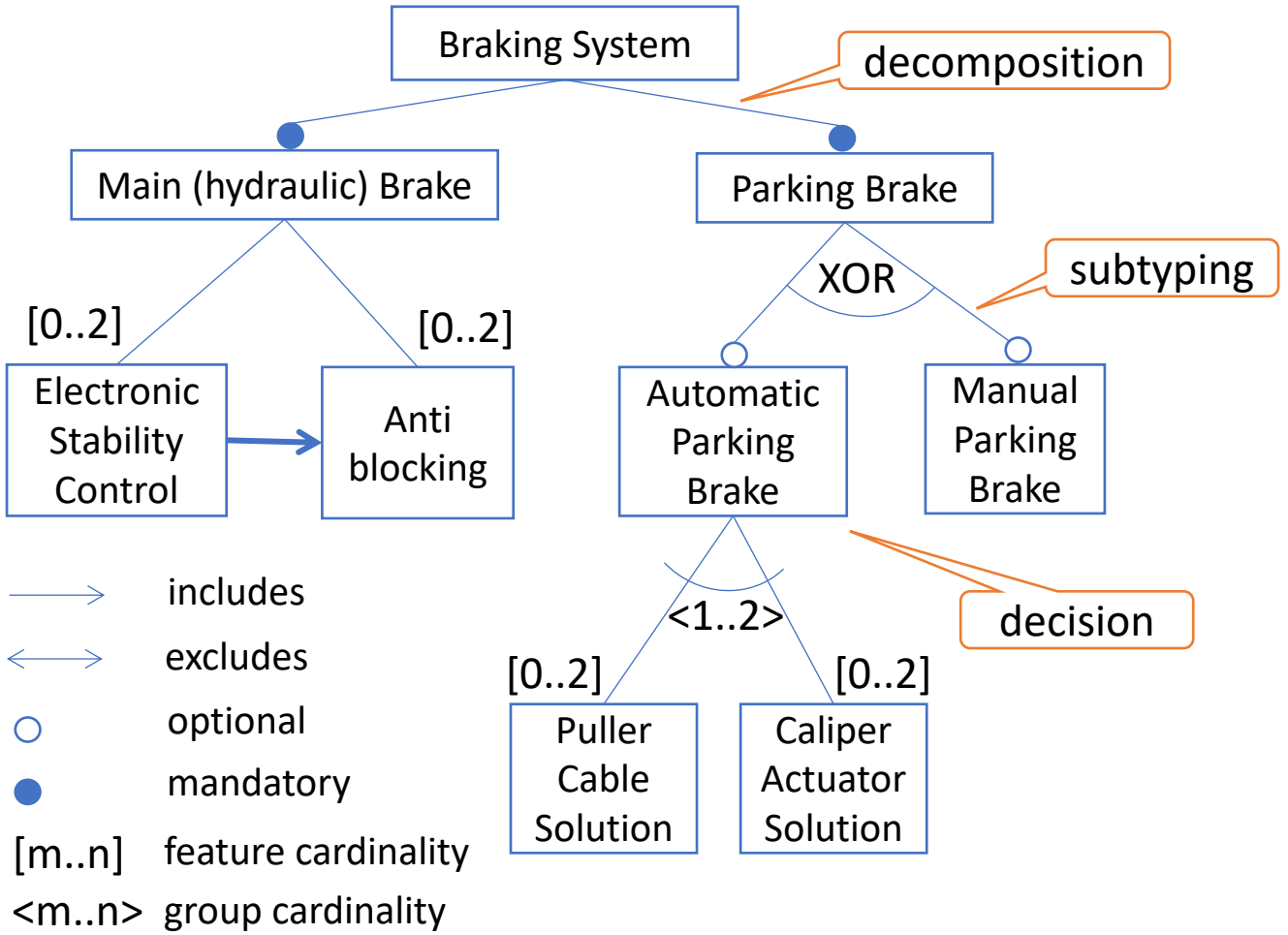
Some problems of the FMs

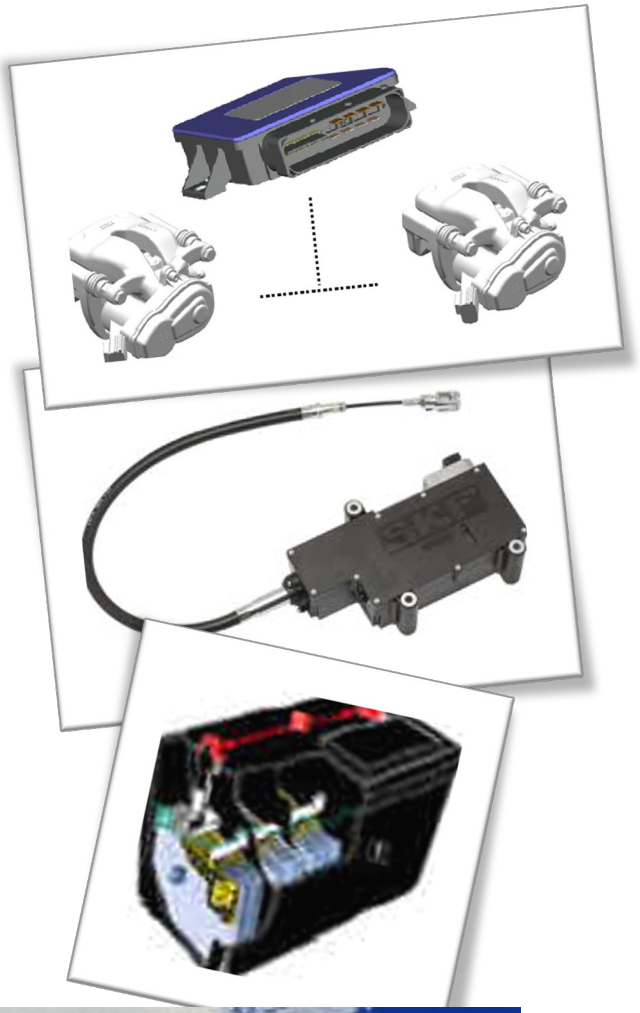






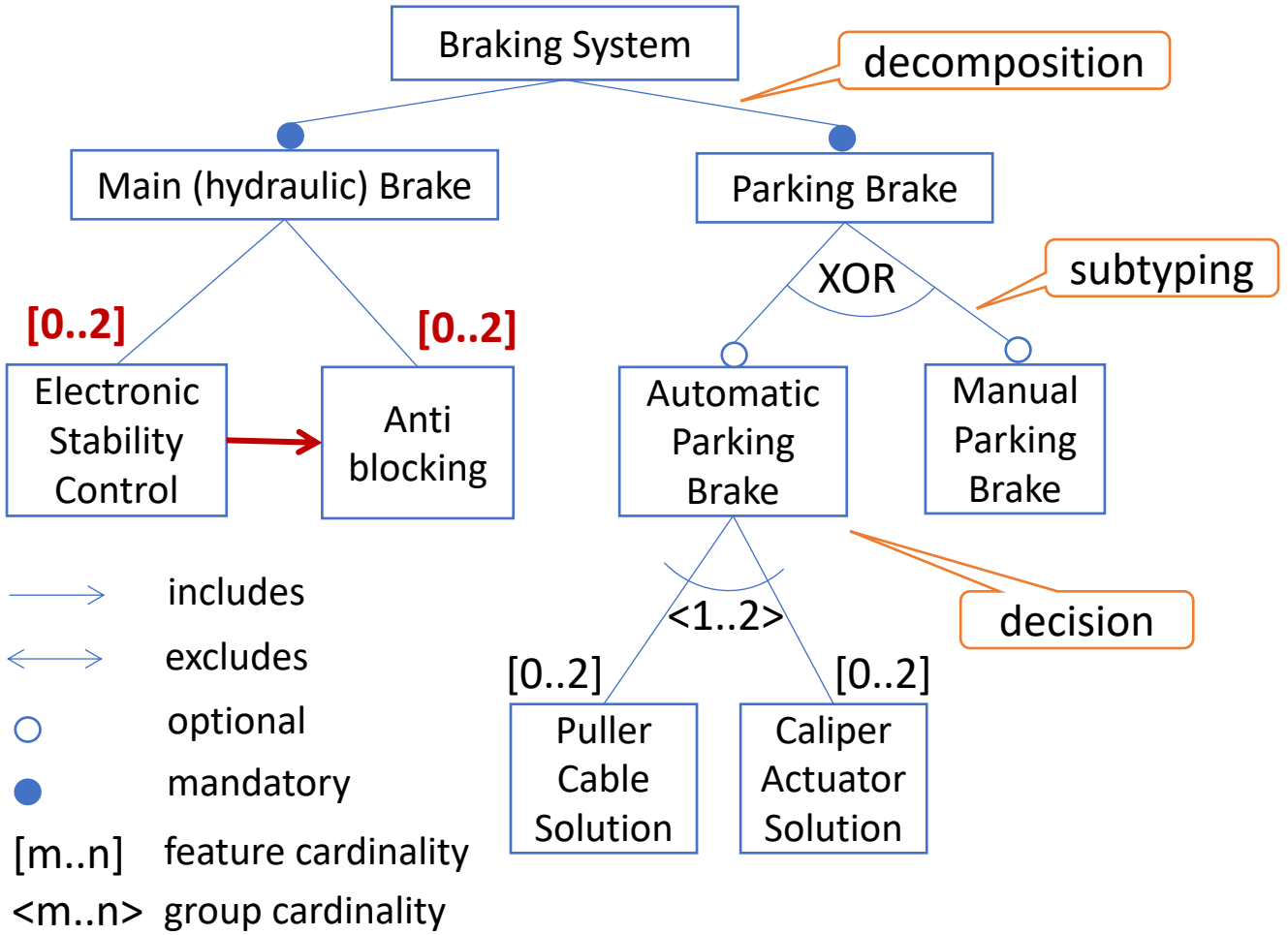
Some problems of the FMs

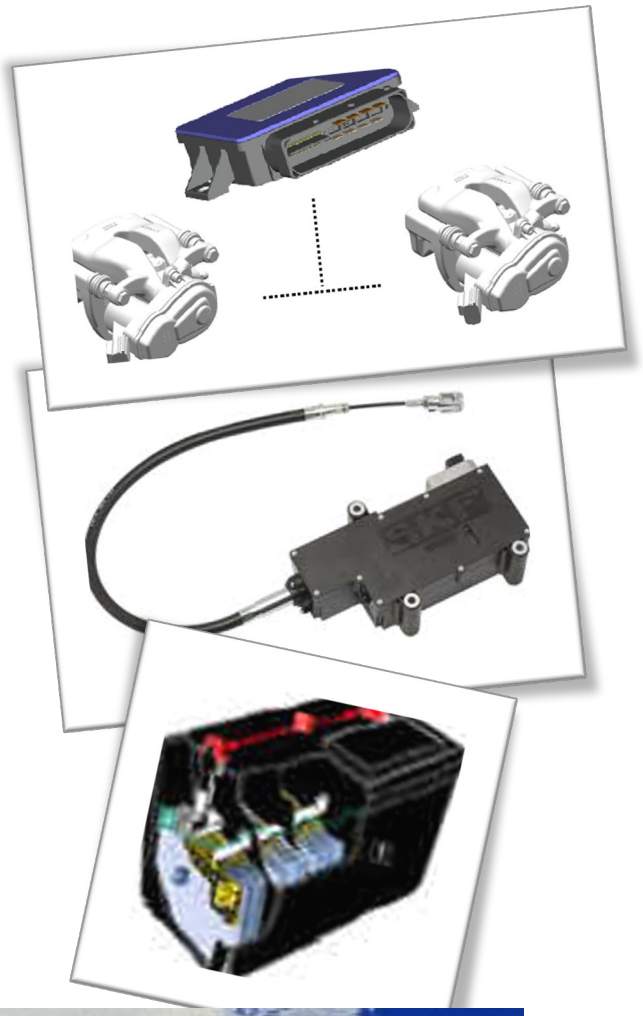






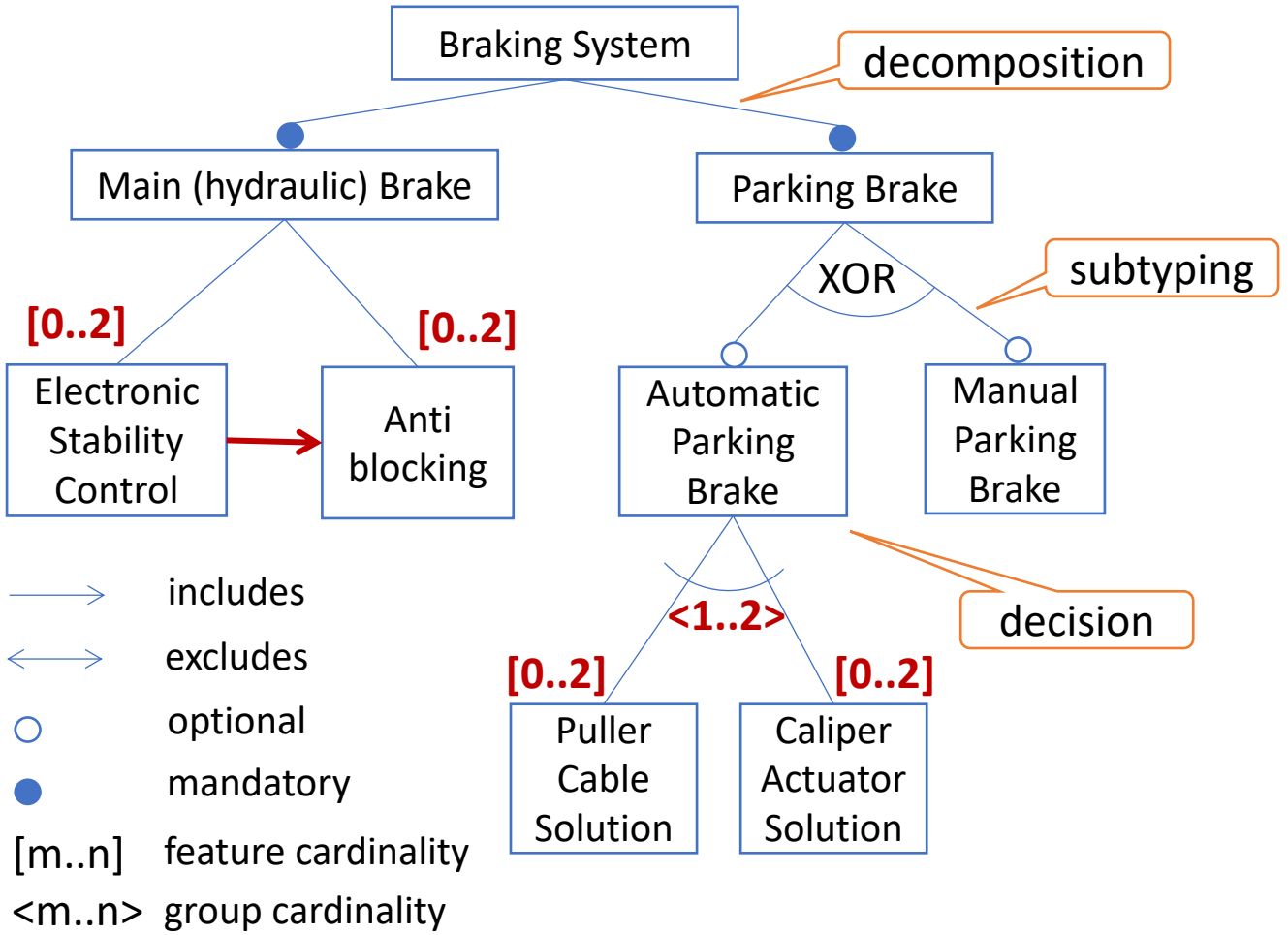
Some problems of the FMs

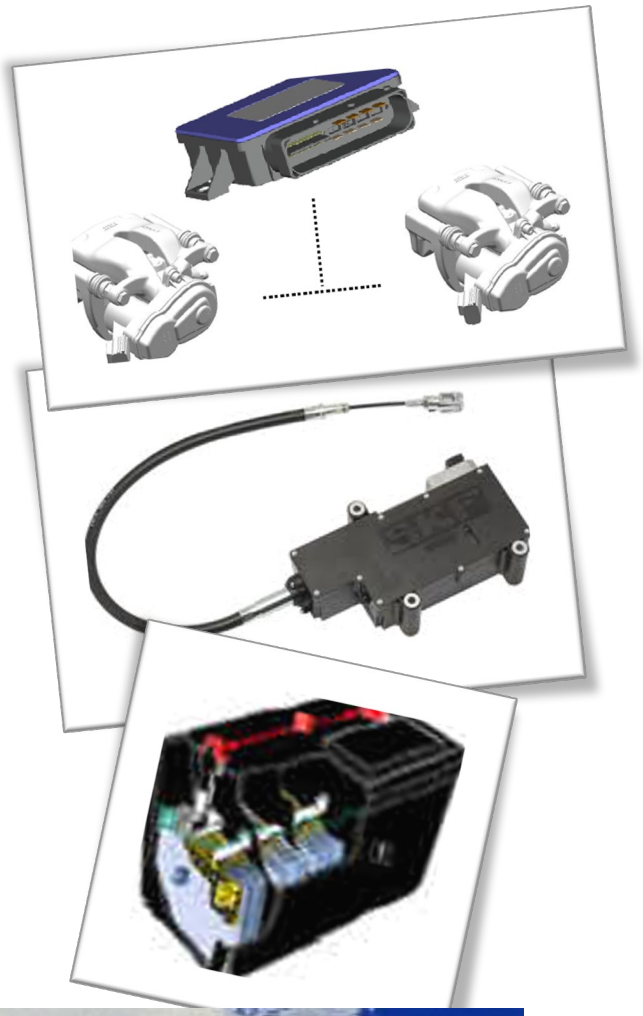






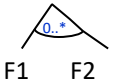
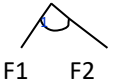
Some problems of the FMs



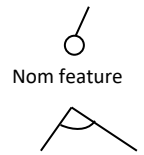
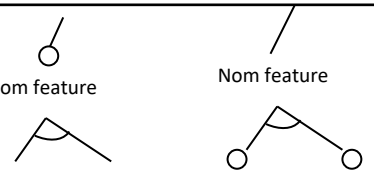
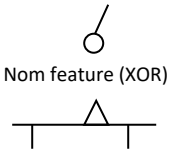
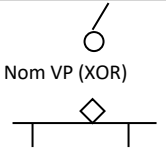
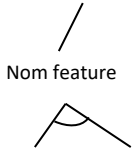
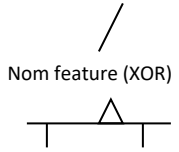
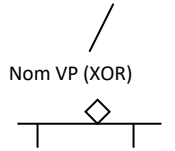
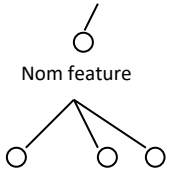
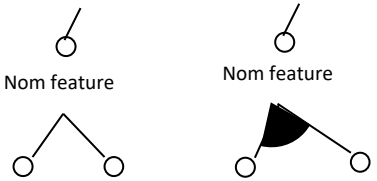
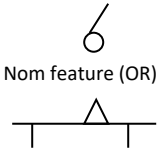
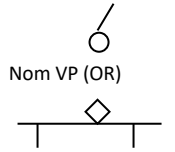
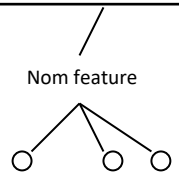
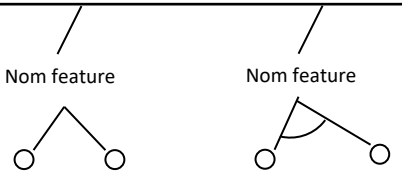
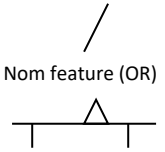
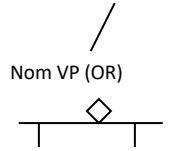
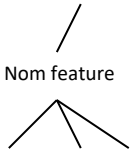
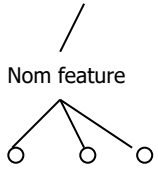
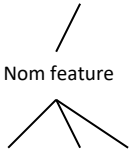
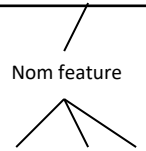




Other feature dialects (basic concepts)

	Type	Mandatory features	Optional features	OR	XOR	Particularities
FODA	arbre			--	--	
FORM	graphe			--	--	Relations d'implémentation, de spécialisation /généralisation
FOPLE	graphe			--	--	Processus (orienté MPP)
FeatuRSEB	arbre	F				Processus (UC)
GP	arbre					Processus (génération)
Bosch	arbre					Feature externe 
FORE	graphe					Relations obligatoires et optionnelles, cardinalités

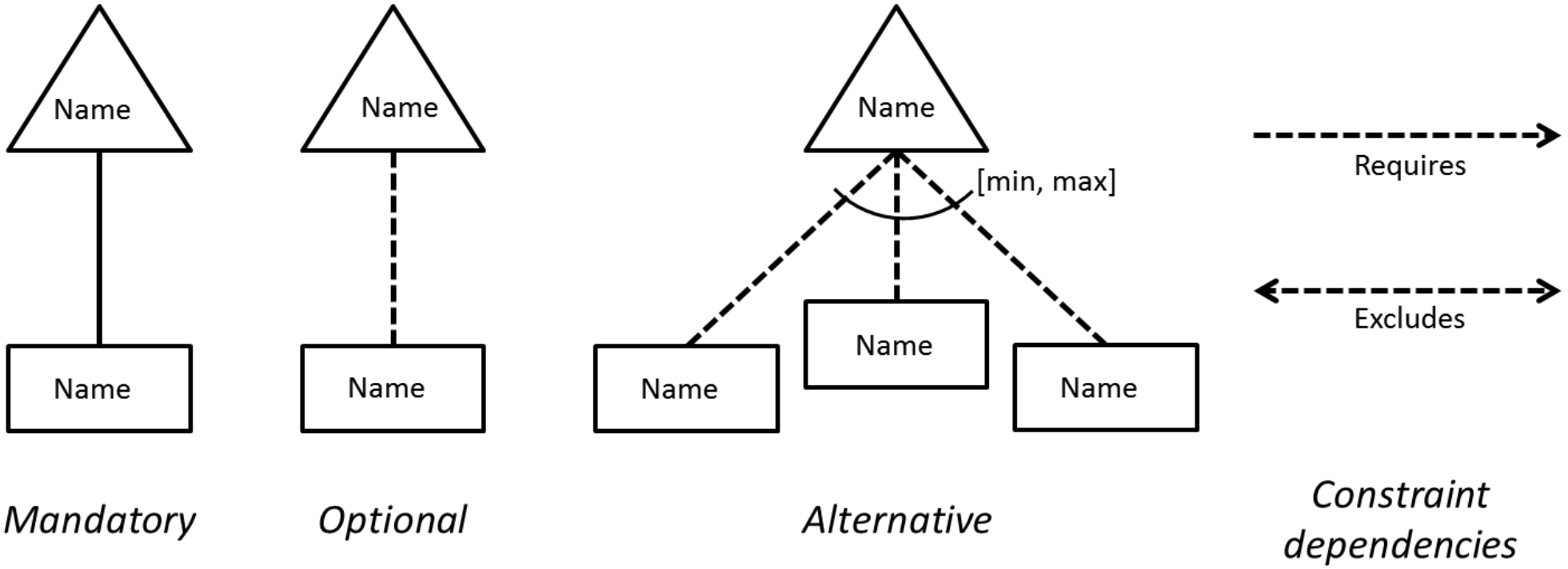
Other feature dialects (cardinalities)

Cardinalities	FODA & FORM	GP	Bosch	FeatuRSEB
0..1				
1				
0..N				
1..N				
N				

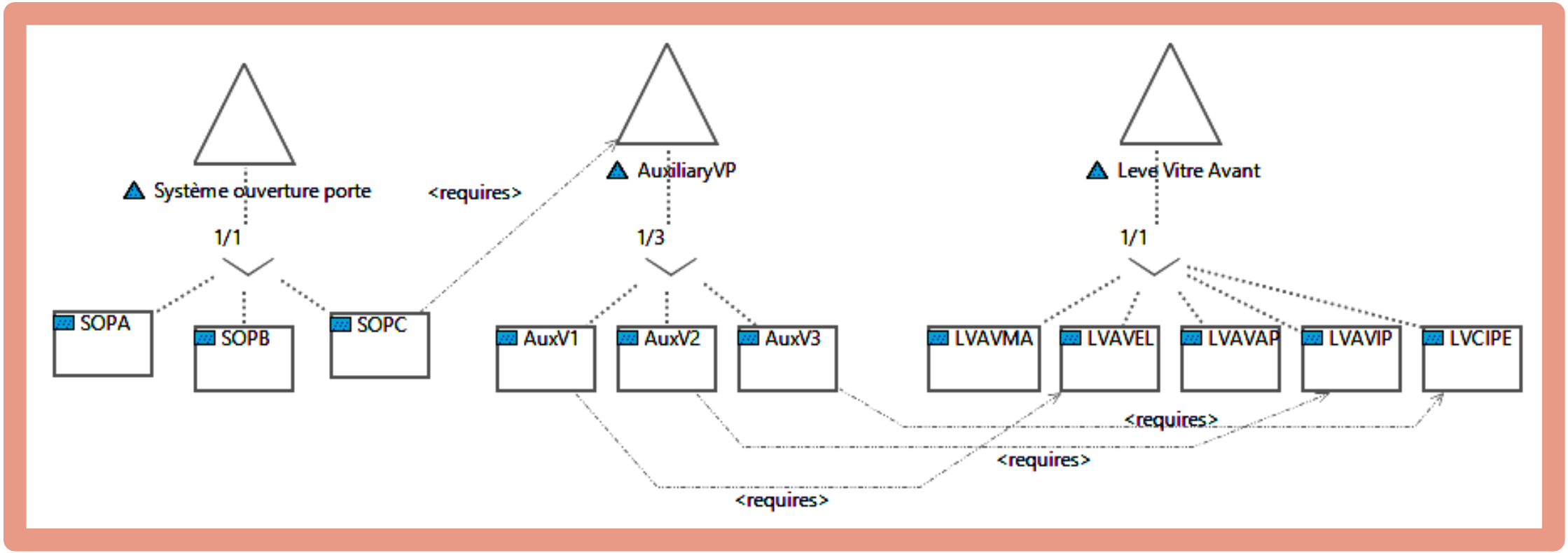
Discussion (FODA dialects)

- Is there any concept, construct or relation type you think is missing from these types of models? Why?
- Do you believe these concepts are sufficient to express any product line? Why or why not?
- These models are by far the most popular model type used in industry, why do you think that is?

Orthogonal Variability Model (OVM)

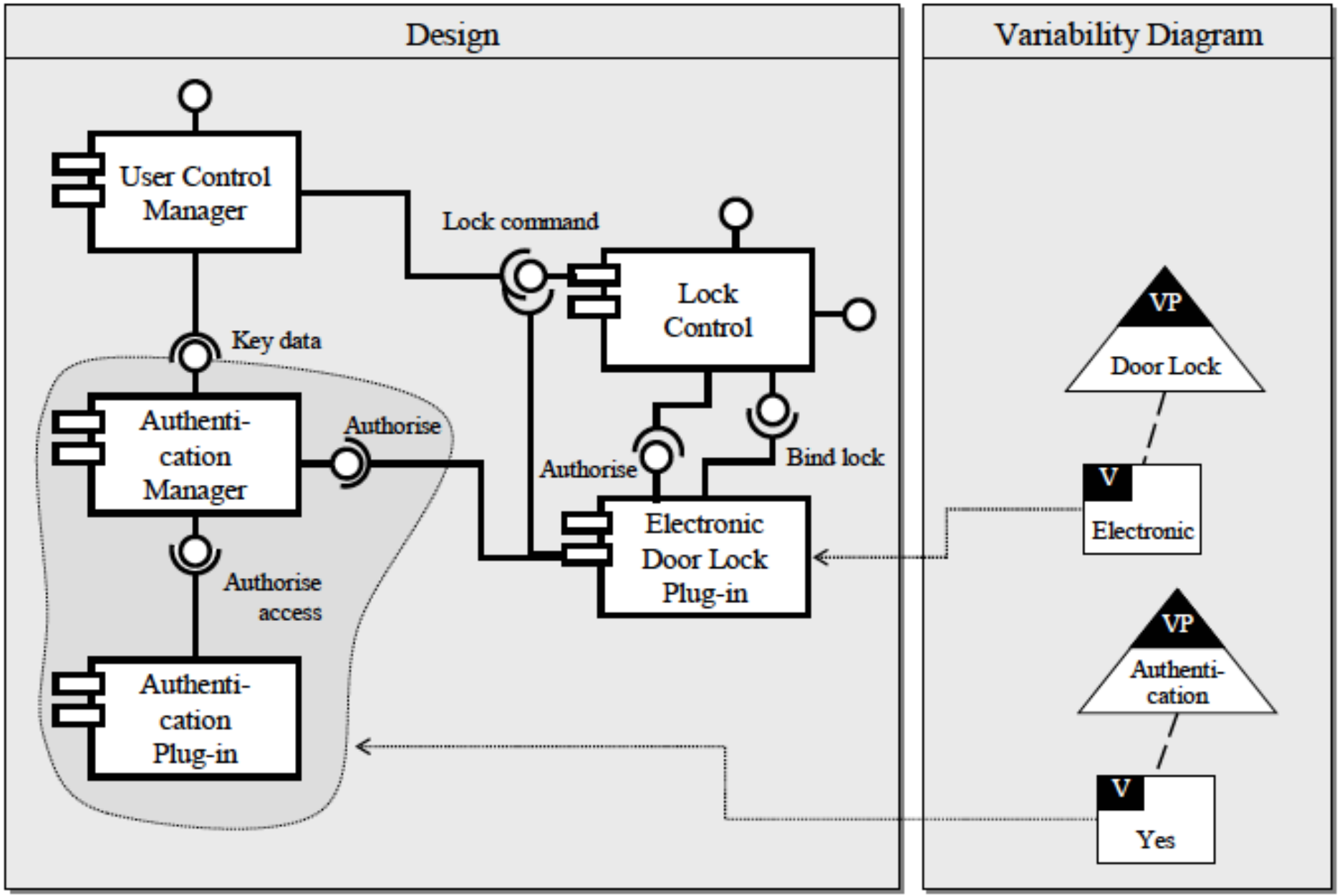


Example of OVM: Using an auxiliary variation point to express constraints



SOPA, SOPB, SOPC : three types of door opening systems
LVAVMA : front manual window lifts
LVAVEL : electric front lifts
LVAVAP : front anti-pinch lifts
LVAVIP : electric impulse lifts
LVCIPE : front electric impulse for driver side and electric for passenger side

Example of OVM: variants know their implementations



(c) Polh et al. 2010

With the OVM language you do not always get well-structured models

Customer visible variability corresponds to the variability stemming from the vehicle level by the product division.

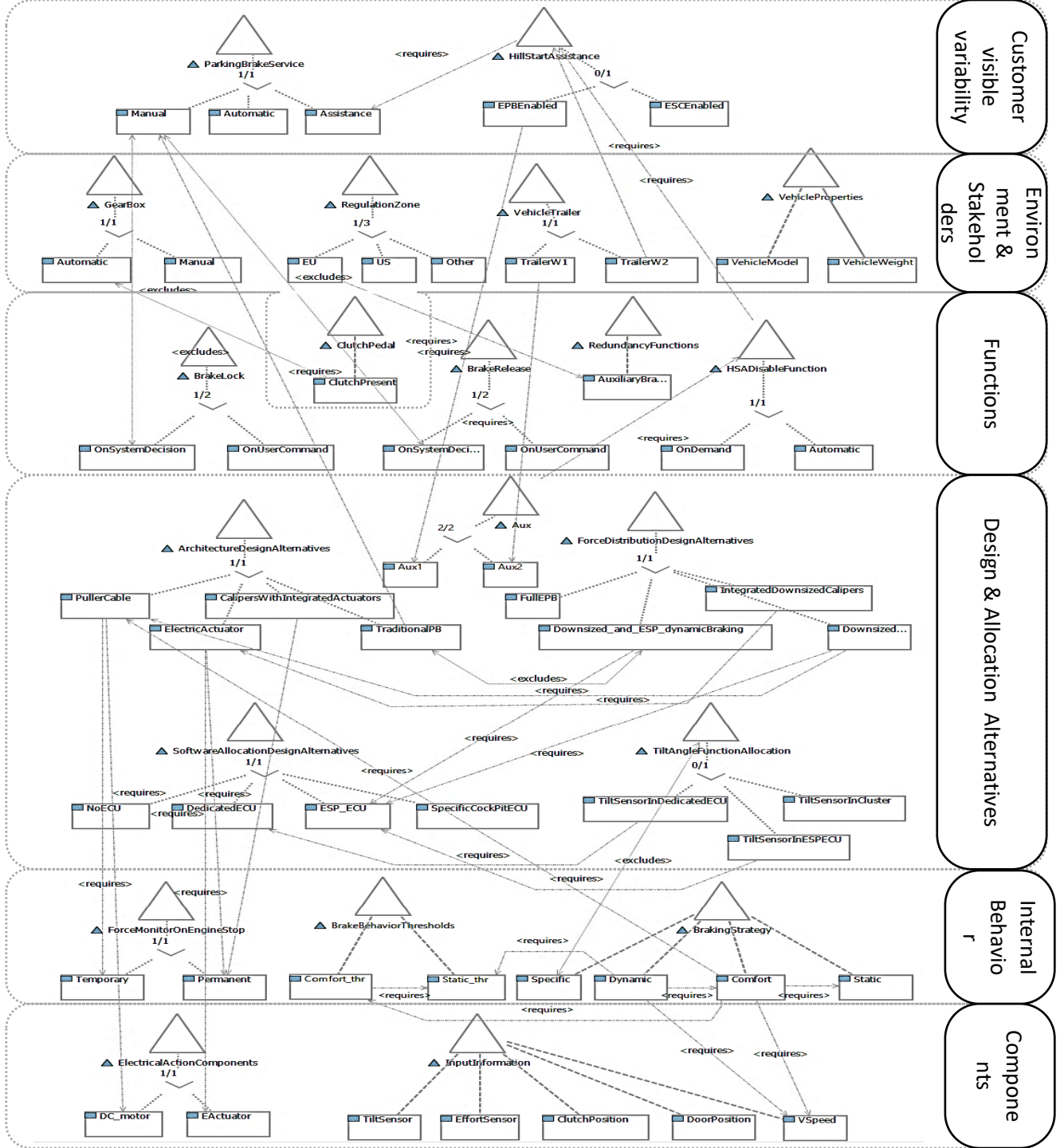
The context viewpoint contains different facets of the system context, such as: system boundary variability, enabling systems and vehicle environment.

The design alternatives viewpoint specifies design decisions that impact the whole or parts of the technical solution.

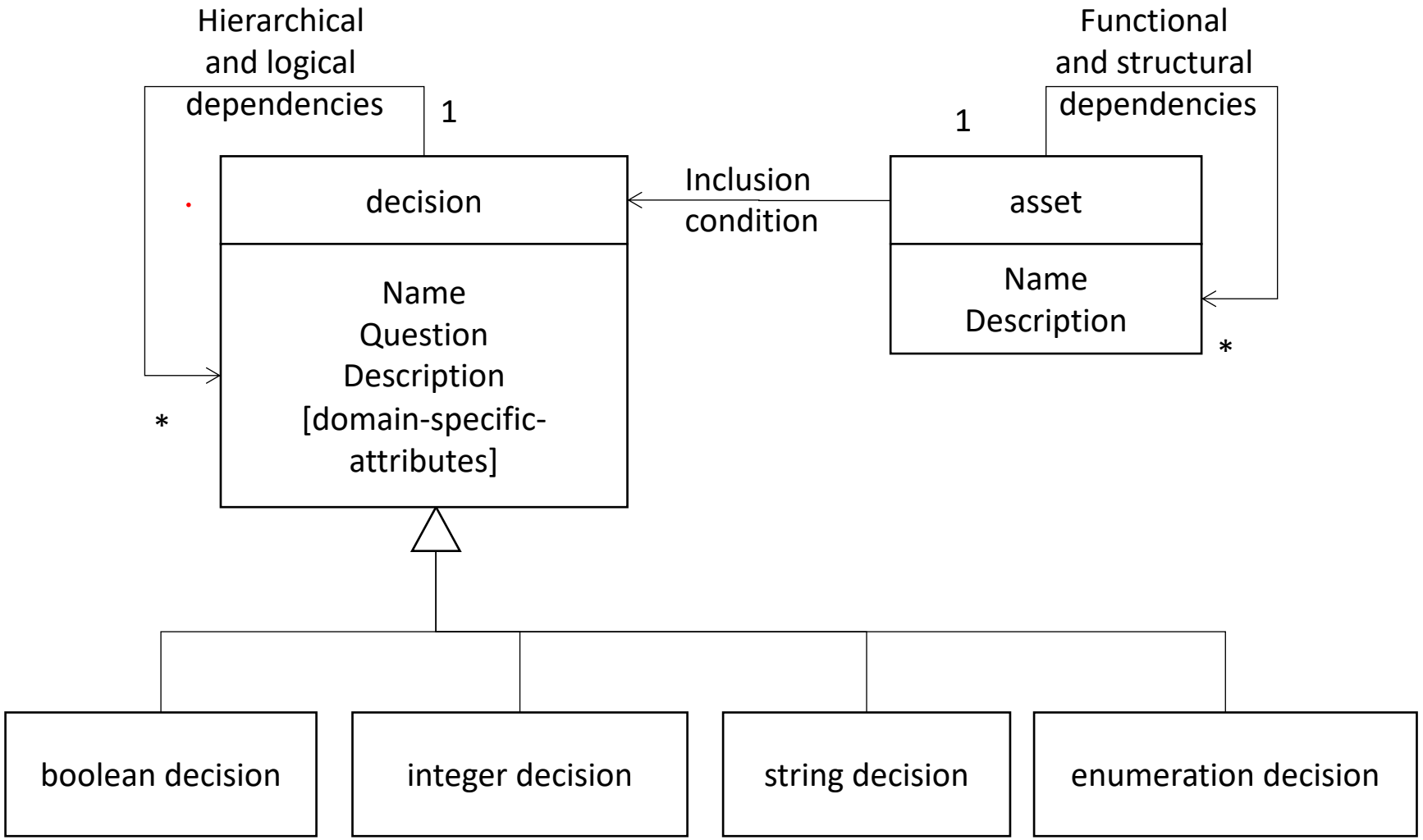
The variability entailed by the system internal behavior viewpoint impacts the states and transitions of the system physical and software components.

The variability entailed by the physical architecture specifies variability in component decomposition, through optional or replaceable components, as well as physical interfaces variability between components.

(c) Dumitrescu et al. 2013



The DOPLER language



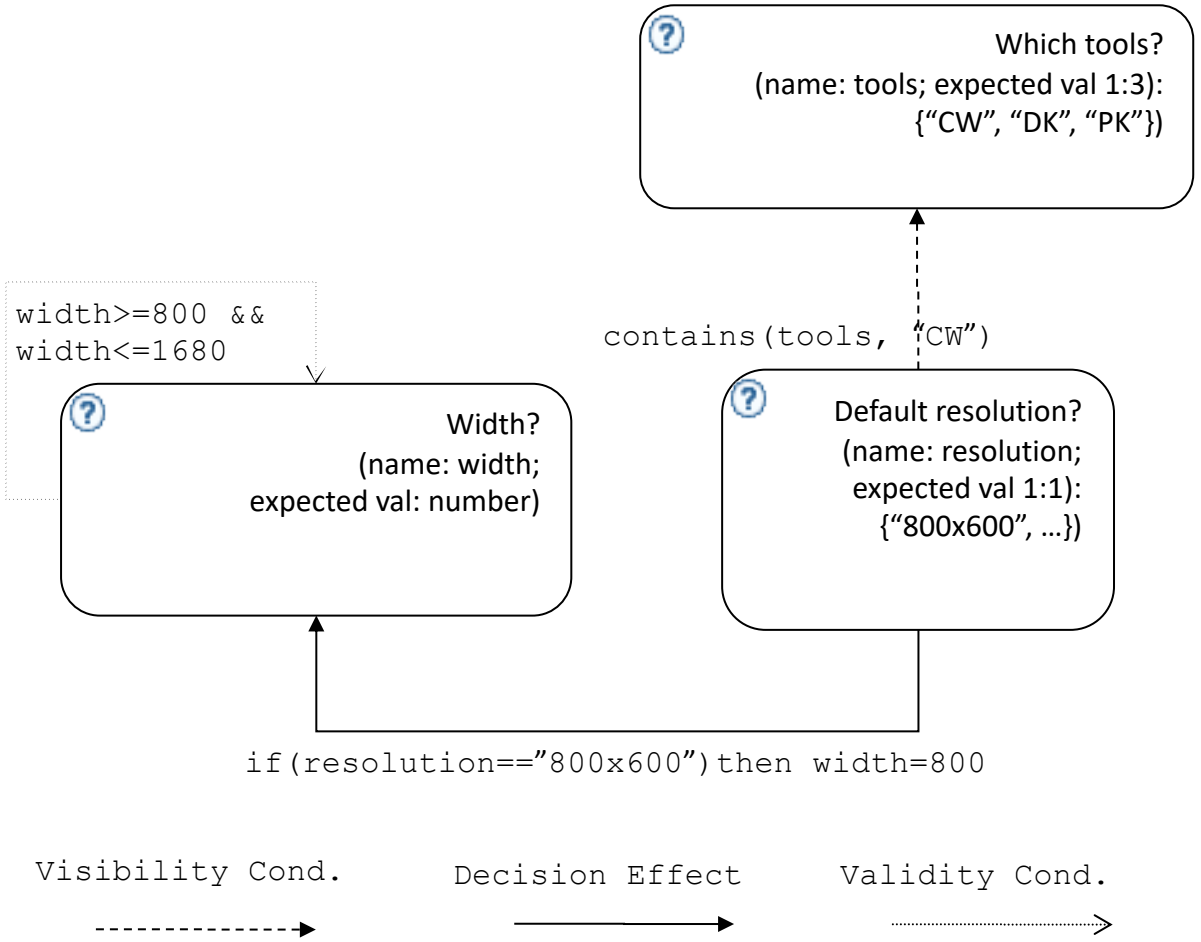
DOPLER Decisions

- Decisions can be of four types: Boolean, Integer, String or Enumeration.
- **DOPLER decision** and **its decision options** can be seen as variables.
- **Expected value** is used as a cardinality over decision options.



Which tools?
(name: tools; expected val 1:3):
{“CW”, “DK”, “PK”})

DOPLER decision relationships

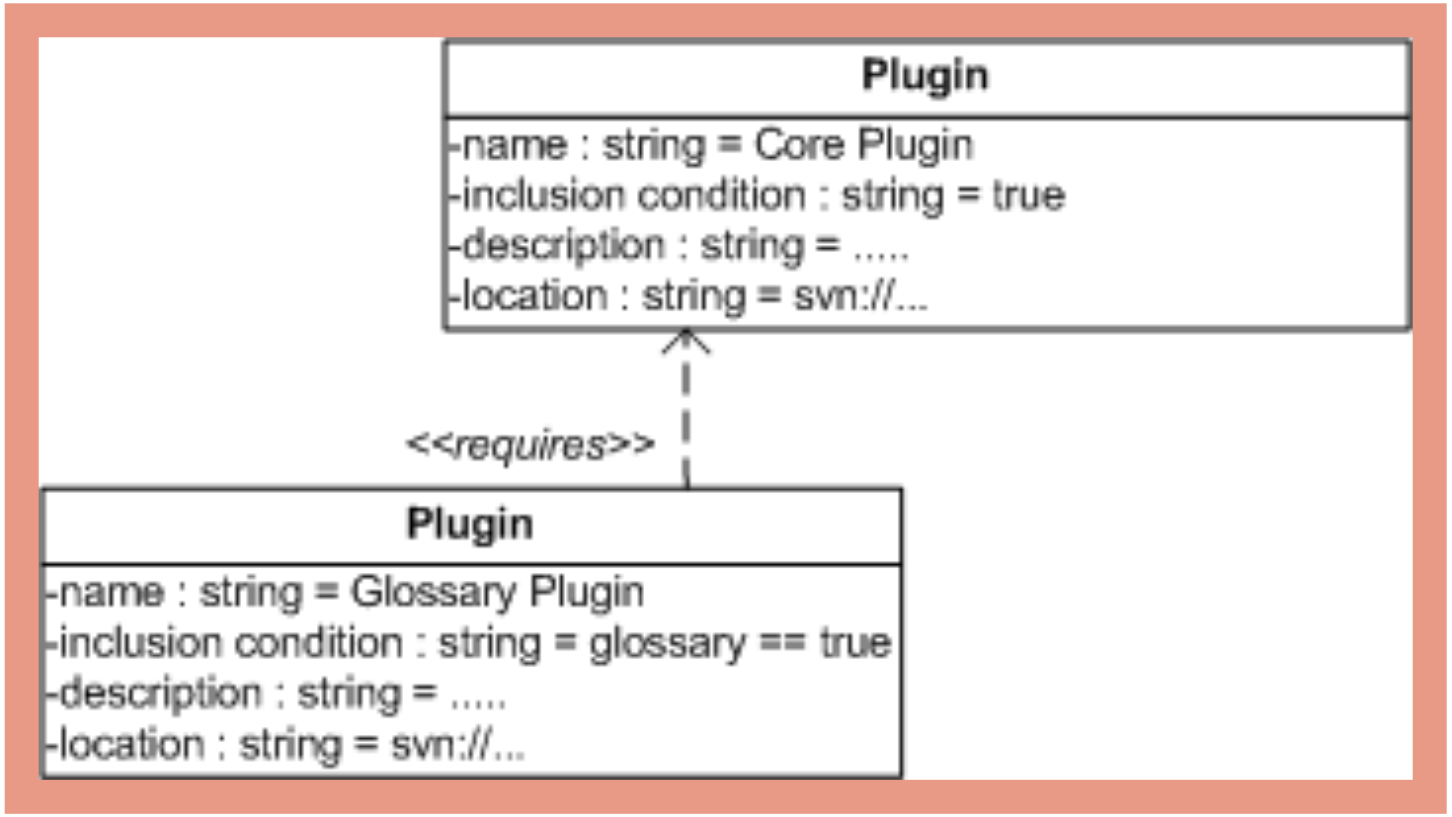


Decisions are related by means of hierarchical and logical dependencies

Visibility conditions and decision effects can be seen as implications

Validity condition restrict the domain of the corresponding

DOPLER Assets



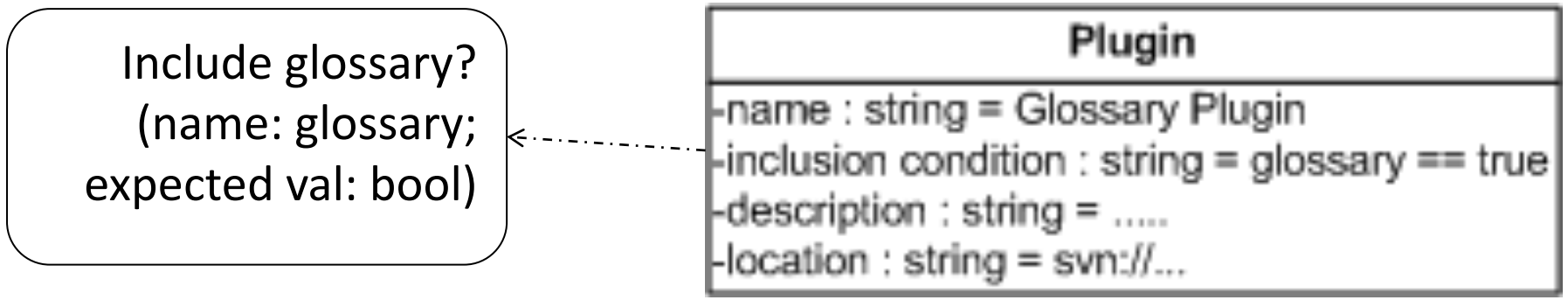
Assets are related by means of functional and structural dependencies

- Requires
- Excludes

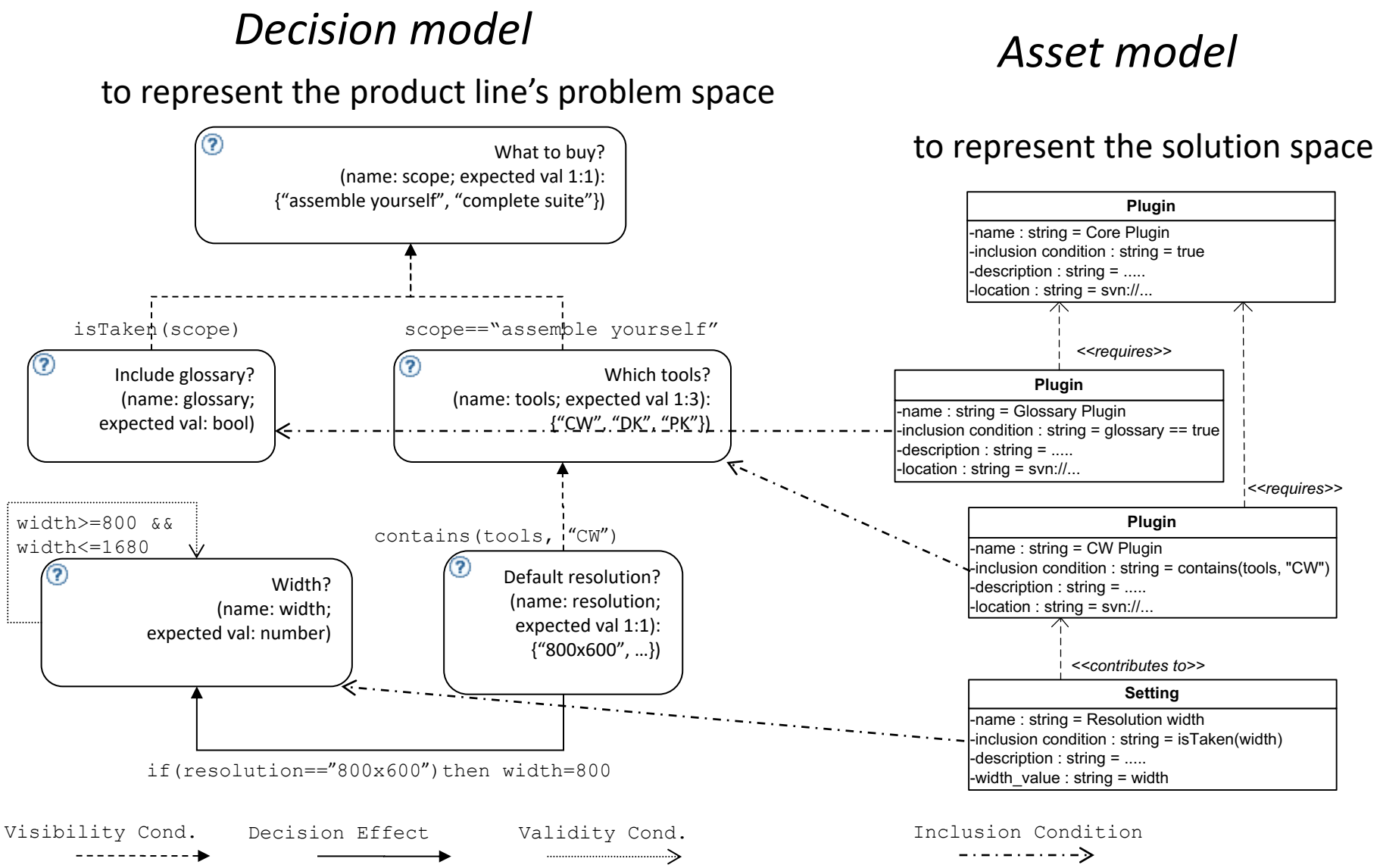
DOPLER inclusion conditions

DOPLER asset can be seen as boolean variables.

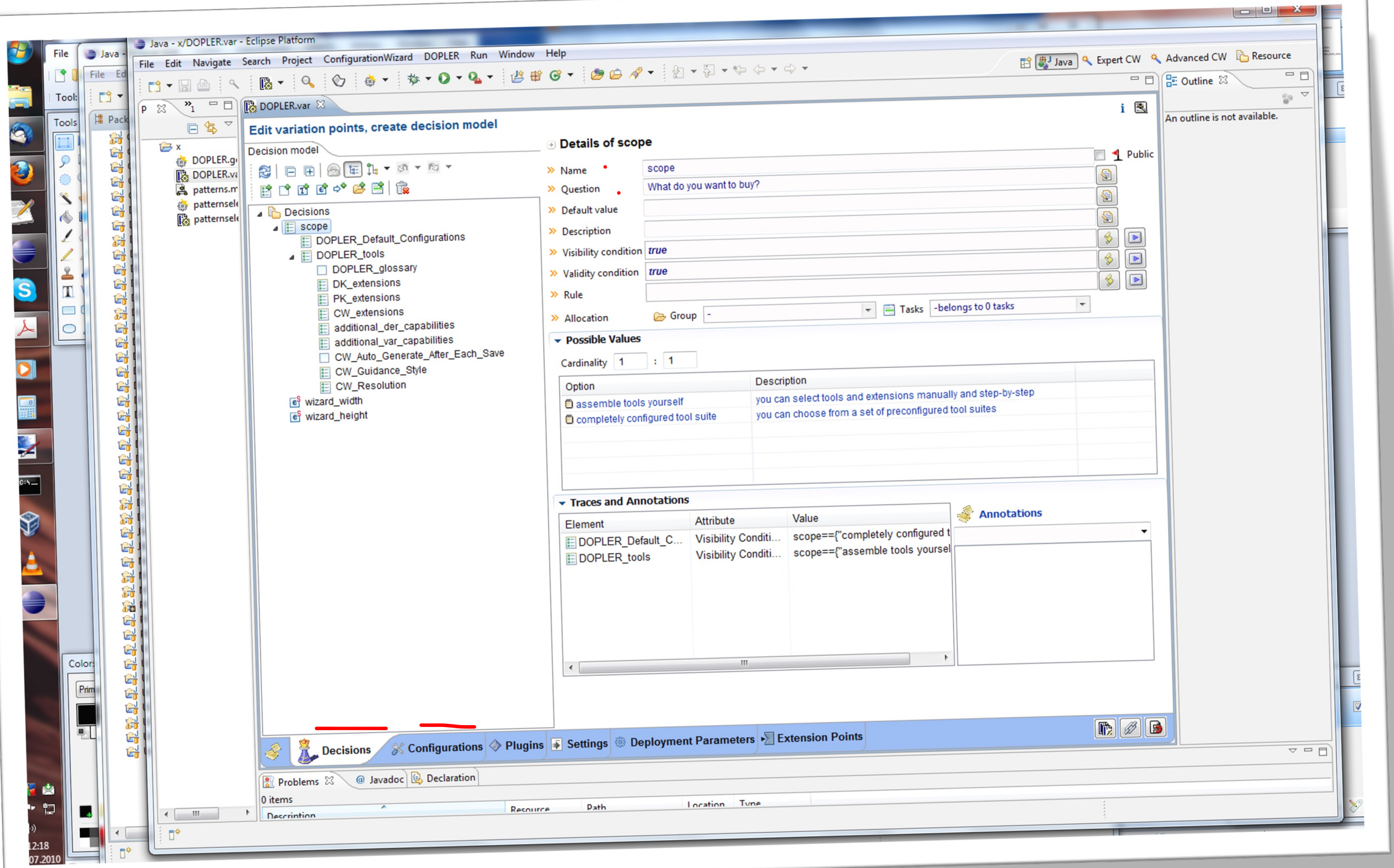
Decisions and assets are linked with **inclusion conditions** defining traceability from the solution space to the problem space



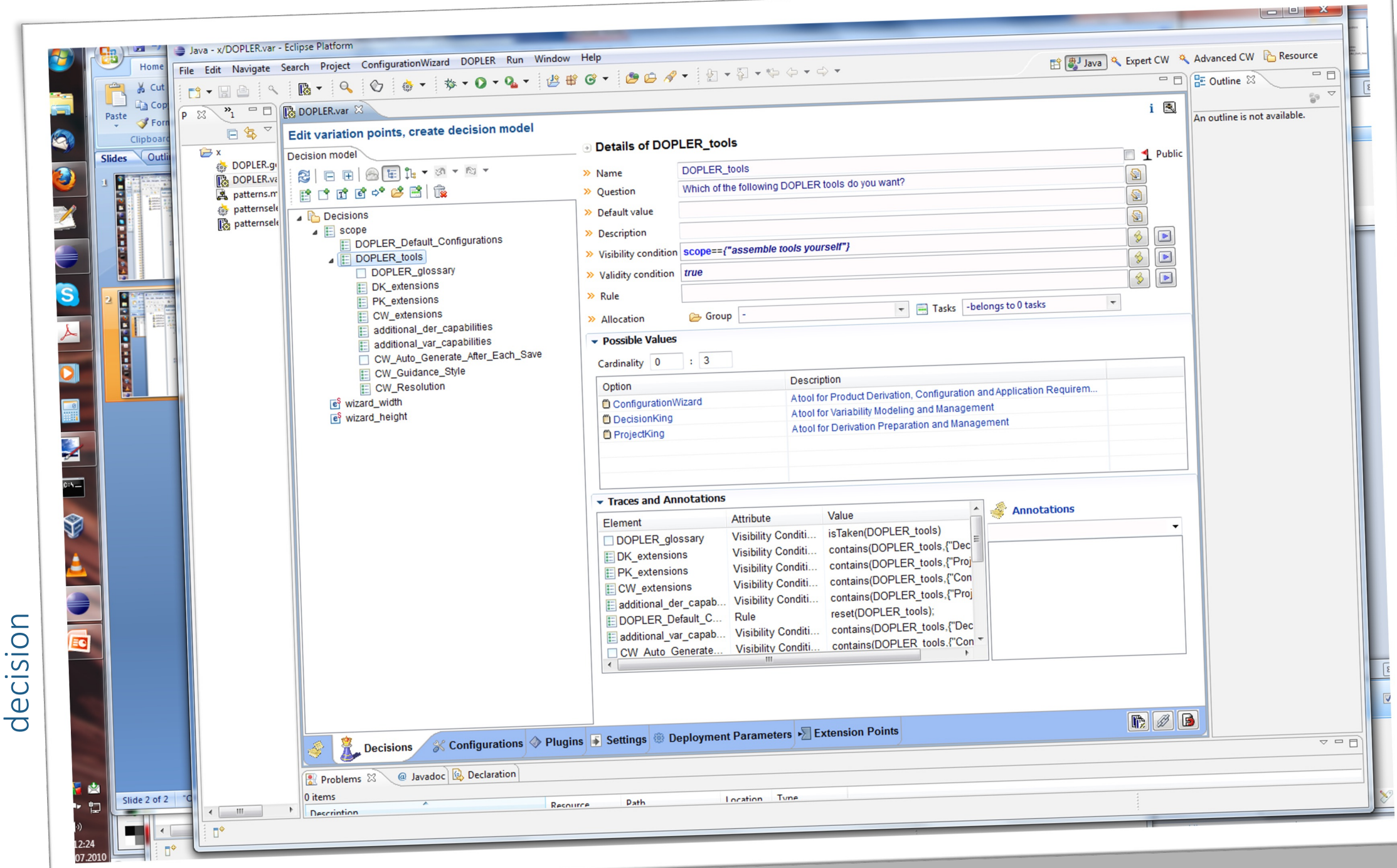
DOPLER variability model



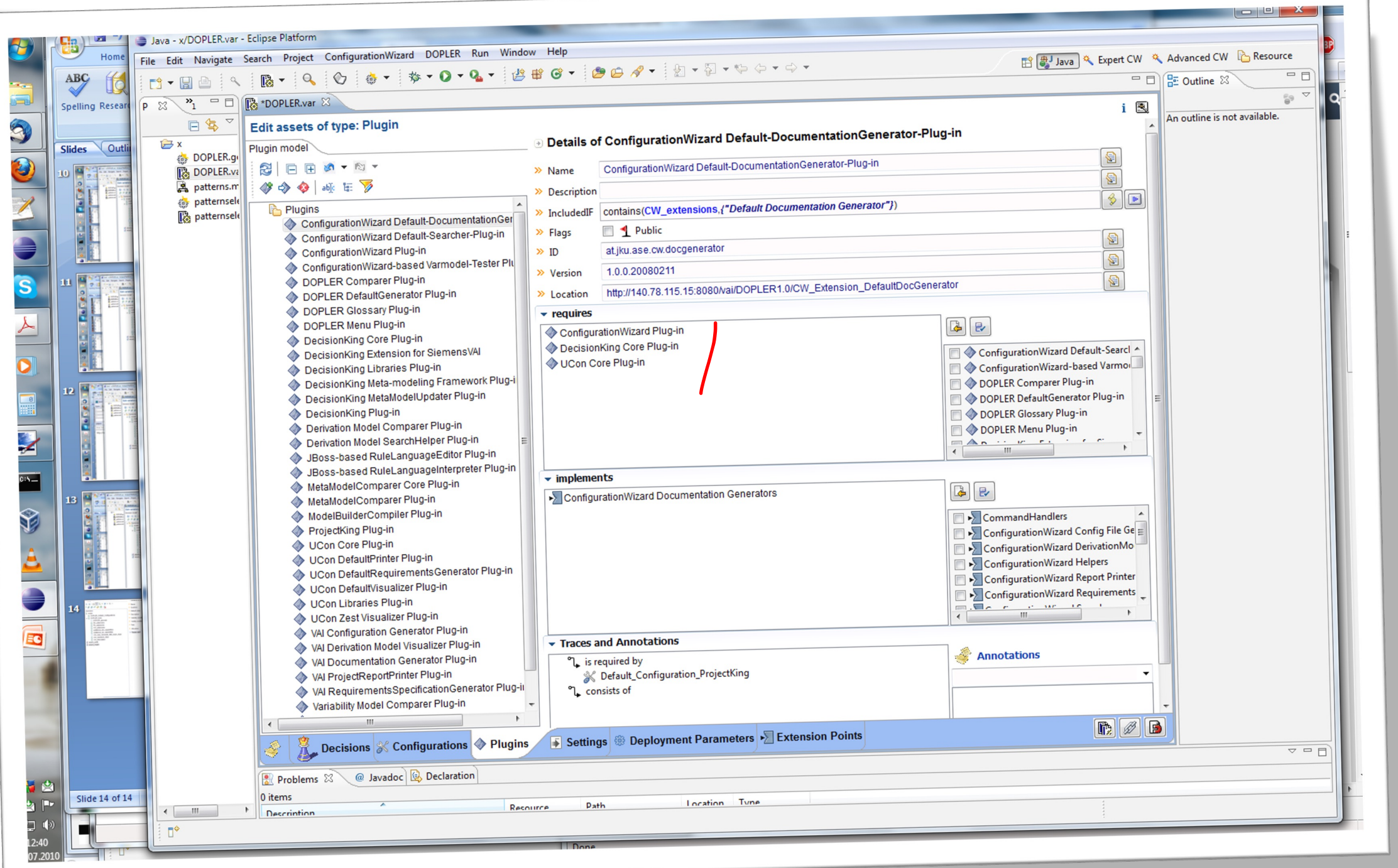
Scope decision



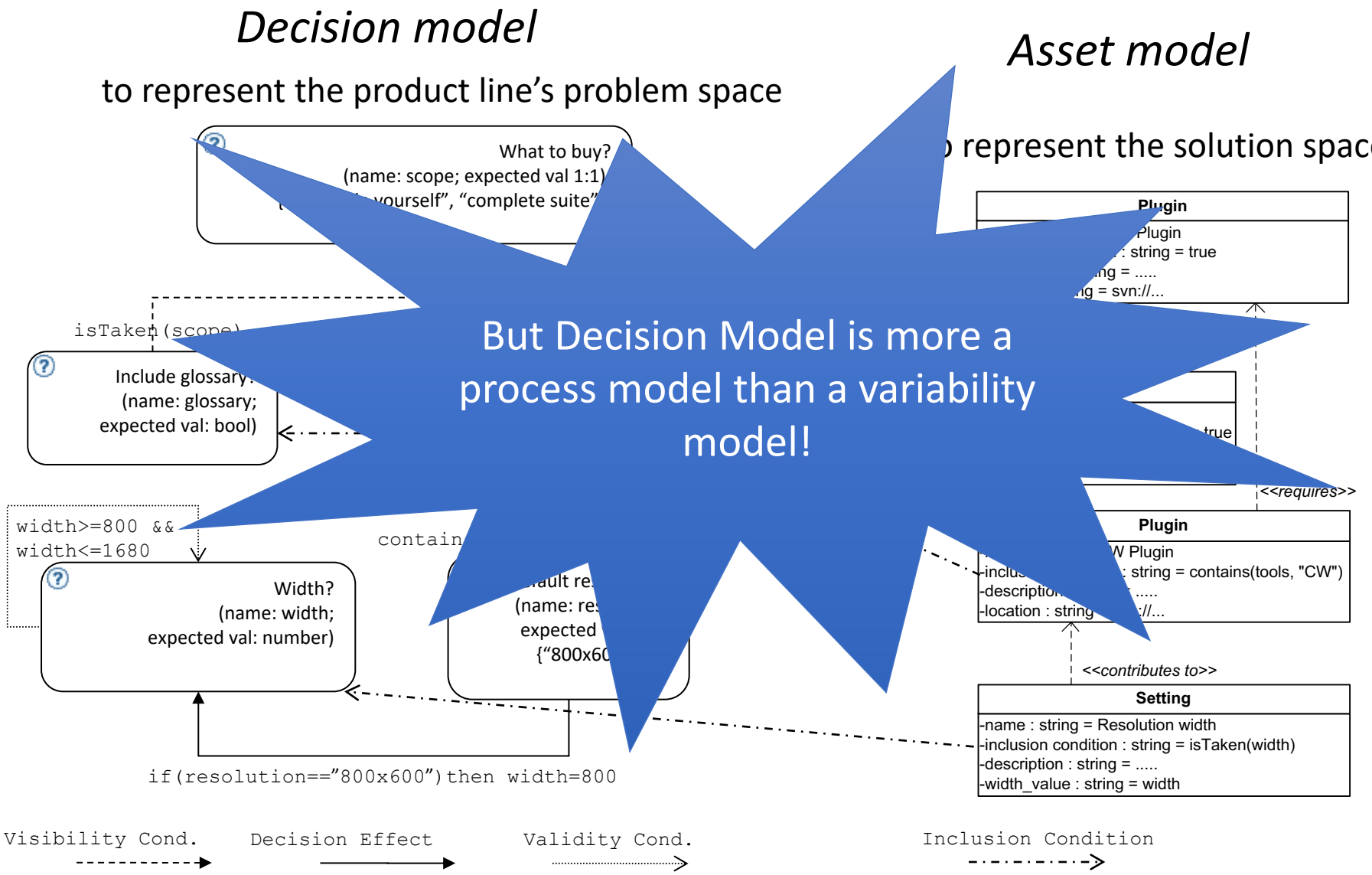
Dopler_tools decision



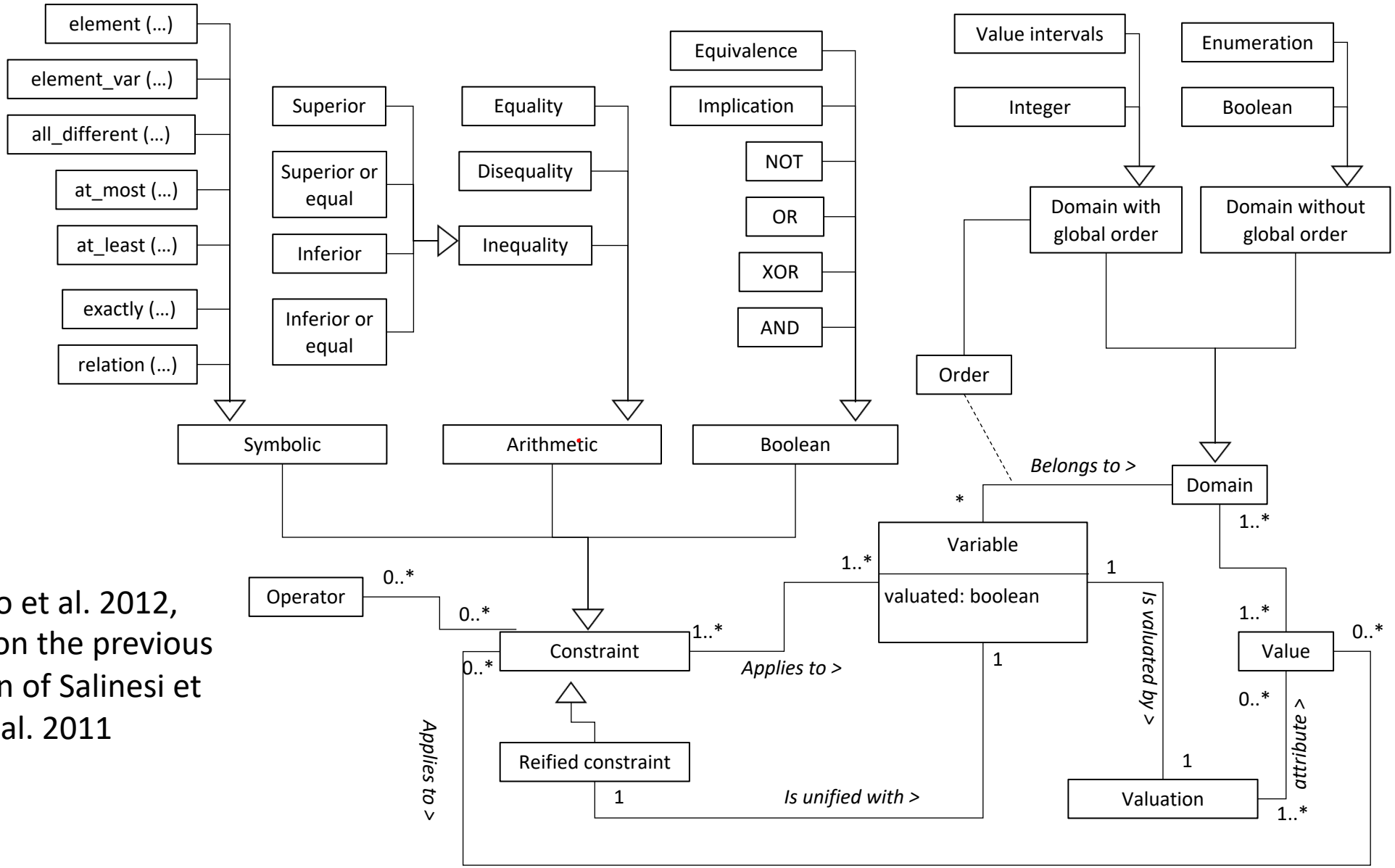
DefaultDocumentationGenerator asset



DOPLER variability model



Constraint-based product line language

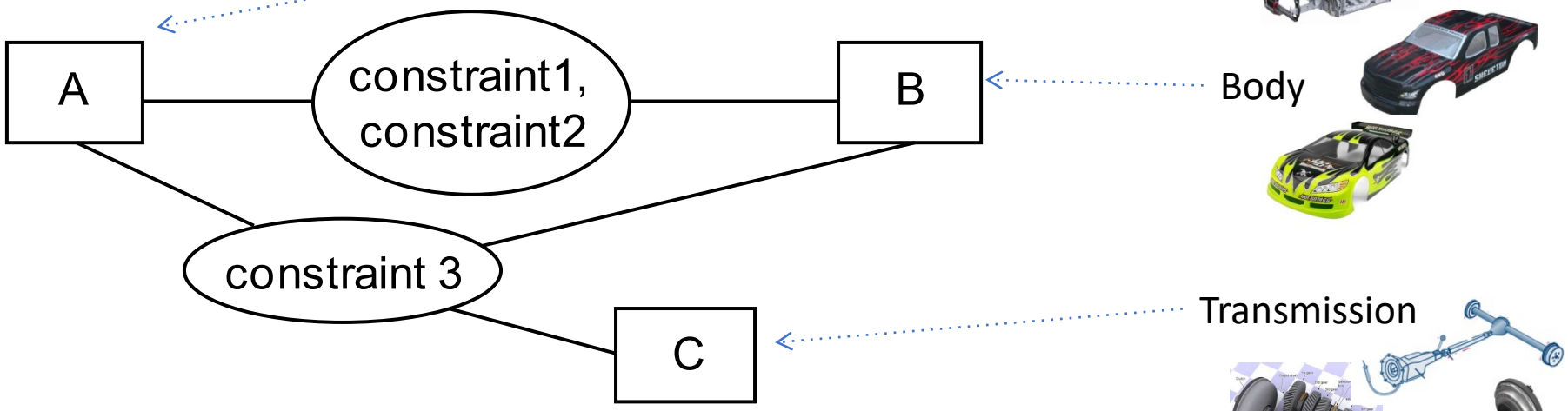


Mazo et al. 2012, based on the previous version of Salinesi et al. 2011

Example of PLM as a contraint program

Problem space

Solution space

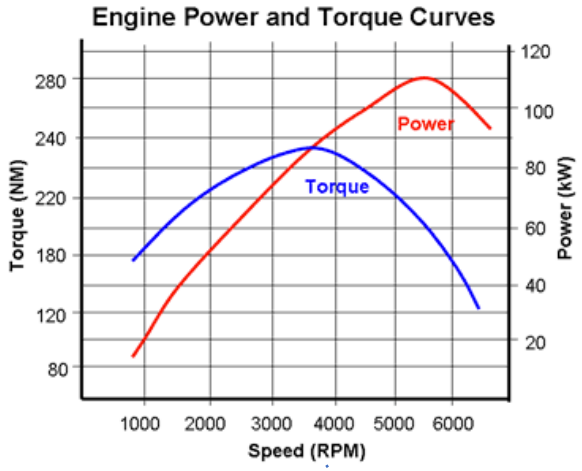


Example of PLM as a contraint program

Context

Problem space

Solution space

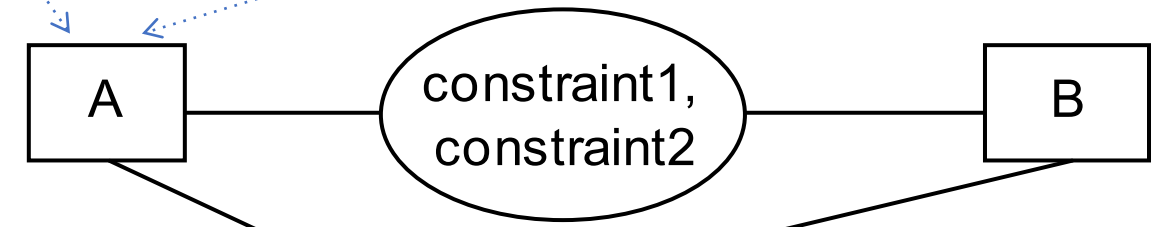




Engine



Body

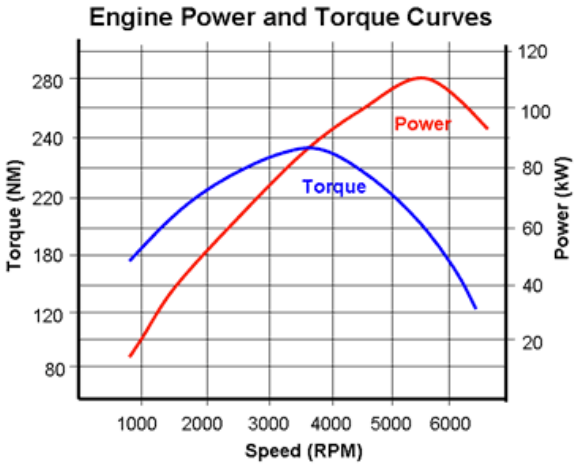


Transmission



Example of PLM as a contraint program

Context



Problem space

```
Variables: domain([A, B], [0, 1]),
           domain([C], [1, 2]),
Constraint1 : B ➔ A,
Constraint2 : at_least([A, B], 1),
Constraint3 : A ≤ B + C ≤ A * 2
```

Solution space

Engine



Body

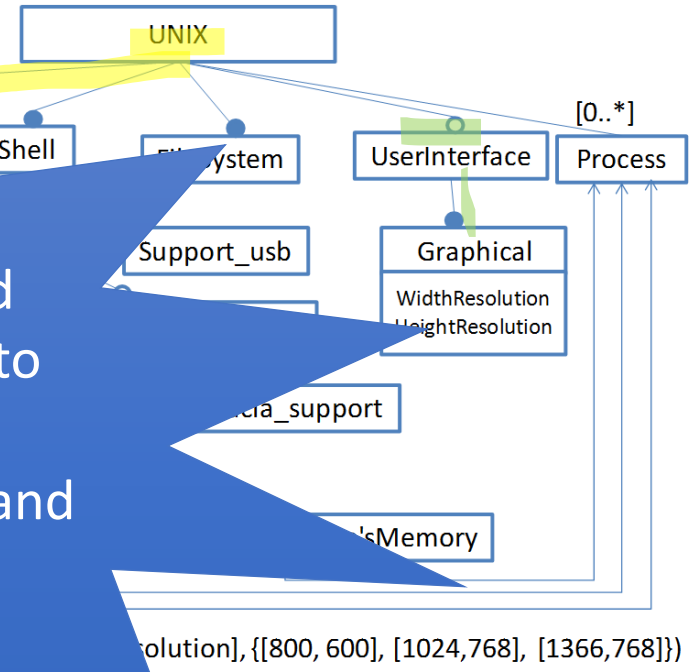


Transmission



Example of PLM as a contraint program

```
[UNIX, Kernel, Scheduling, ExecutingInstructions, InterpretingInstructions,
AccomplishingTheTransferOfData, AllocatingTheMachinesMemory, Shell, FileSystem,
UserInterface, Graphical, Process] ∈ {0, 1} ∧
[WidthResolution] ∈ {0, 800, 1024, 1366} ∧ [HeightResolution] ∈ {0, 600, 768} ∧
[Support_usb, Cdrom_atech, Pcmacia_support] ∈ {0,1,2} ∧
[BoolSupport_usb, BoolCdrom_atech, BoolPcmacia_support] ∈ {0, 1} ∧
UNIX = 1 ∧
UNIX = Kernel ∧
Kernel = Allocati TheMachinesMemory ∧
AllocatingTheMachin ory ⇒ Process ∧
Kernel = Scheduling ∧
Scheduling ⇒ Process ∧
Kernel = AccomplishingThe
AccomplishingTheTransferOfDa
Shell ⇒ InterpretingInstructi
Kernel = Interpr
Shell ⇒ Executi
Kernel = Executing
Support_usb ⇔ BoolSupp
Cdrom_atech ⇔ BoolCdrom_a
Pcmacia_support ⇔ BoolPc
0 ≤ BoolSupport_usb +
UNIX = Shell ∧
UNIX = FileSystem ∧
UNIX ≥ UserInterface ∧
UserInterface = Graphical ∧
Graphical=1 ⇔ (WidthResolution= ∧ Height on=H1)∧
Graphical = 0 ⇔ (WidthResolution=0 ∧ Height tion=0)∧
fd_relation([[800,600], [1024,768], [1366,76 [W1,H1])∧
UNIX ≥ Process ∧
R1 = Process1 + Process2 + Process3 + Proces + Process5∧
Process ≤ R1 ≤ Process*5
```



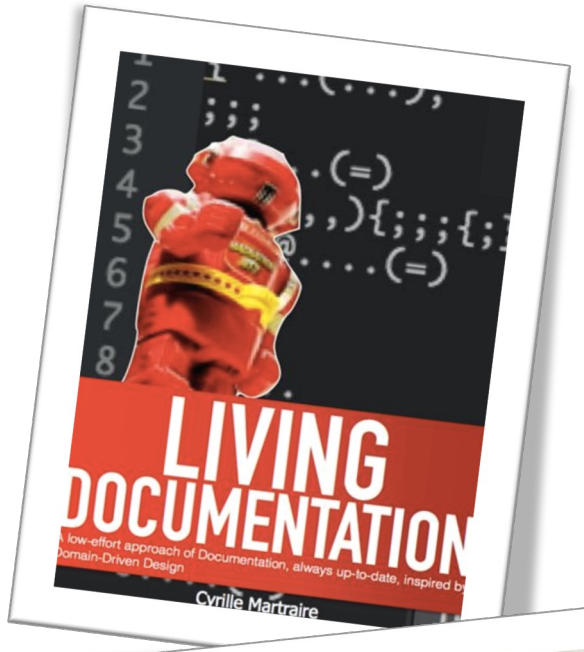
However, the constraint-based representation can be difficult to build (for certain people), understand (for certain people) and to use with stakeholders !



1. **Analysis**
   A. **…**
   B. **Strategies to elicit the requirements of a PL**
      a. **Requirements-driven strategy**
      b. Low-end vs. high-end product strategy (reference-product strategy)
      c. Multiviewpoint strategy

Requirements-driven strategy

1. Start collecting and analysing **the available documentation of the domain**
2. Identify **commonality** (mandatory) (no conditions) ➔ mandatory requirements
3. Identify **variability** (optionality) keywords ➔optional requirements
4. Identify keywords representing **dependencies** ➔ optional requirements + constraints
5. Complete with the documentation about the implementation assests and write the requirements corresponding with a template





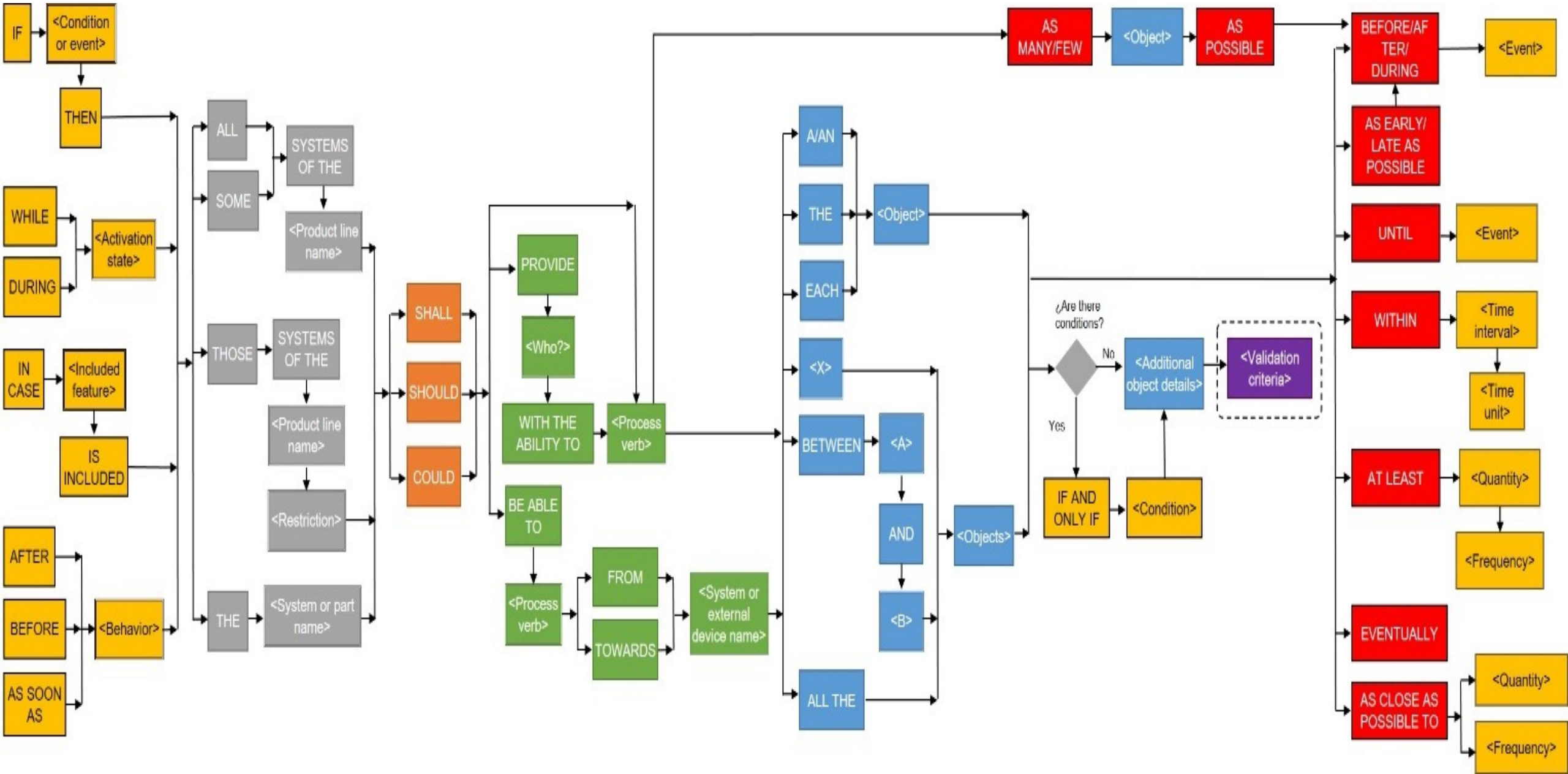
Example of available documentation

- UNIX has one KERNEL.
- Some of the mandatory functions of the KERNEL are:
  - ALLOCATING THE MACHINE'S MEMORY to each PROCESS
  - SCHEDULING the PROCESSES
  - ACCOMPLISHING THE TRANSFER OF DATA from one part of the machine to another
- UNIX can have several PROCESSES (or none) for each user. The collection of PROCESSES varies even when the UNIX product is full-configured. For the sake of presentation, this thesis will consider only five processes.
- UNIX offers a logical view of the FILE SYSTEM. A FILE SYSTEM is a logical method for organising and storing large amounts of information in a way that makes its management easy.
- The KERNEL is composed of static or dynamic software modules. If the kernel was compiled for a specific hardware platform and cannot be changed, it is called a static Kernel. If the Kernel has the ability to dynamically load modules so that it can 'adapt' to a platform, it is called a dynamic Kernel. For instance, the modules SUPPORT_USB, CDROM_ATECH, and PCMCIA_SUPPORT cannot be charged, can be charged in a static way or can be charged in a dynamic way.
- The SHELL is a command interpreter; it takes each command and passes it to the KERNEL to be acted upon.
- The GRAPHICAL interface is characterized by a WIDTH RESOLUTION and a HEIGHT RESOLUTION that can have the following couples of values [800,600], [1024,768] and [1366,768].

Each requirement can be fully documented with the following table (adapted from (IEEE 830, 1998))

<id>	Requirement descriptive name
Version	<version number> (version creation date)
Description	<use a template to specify the domain requirement in semi-structured natural language, e.g., Mazo et al. 2020>
Function/module	<functionality or module to which the requirement belongs>
Type	<indicate here: FR, NFR, legal requirement, change requirement, technical requirement, etc.>
Variability	<specify if the requirement is optional or mandatory and the alternatives that should be supported by this requirement>
Priority	<Priority of the requirement in the product line>
Delivery date	<indicate here the expected/needed delivery date>
Stability	<risk of requirement evolution>
Source	<role and name of contributors>

Template to the requirements specification of systems and families of (self) adaptive systems (Mazo et al., 2020)



[Condition or event], **ALL|SOME** SYSTEMS OF THE <Product line name> <Priority> ...
[Condition or event], **THOSE** SYSTEMS OF THE <Product line name> <Restriction> <Priority>...
... (see template for more options)

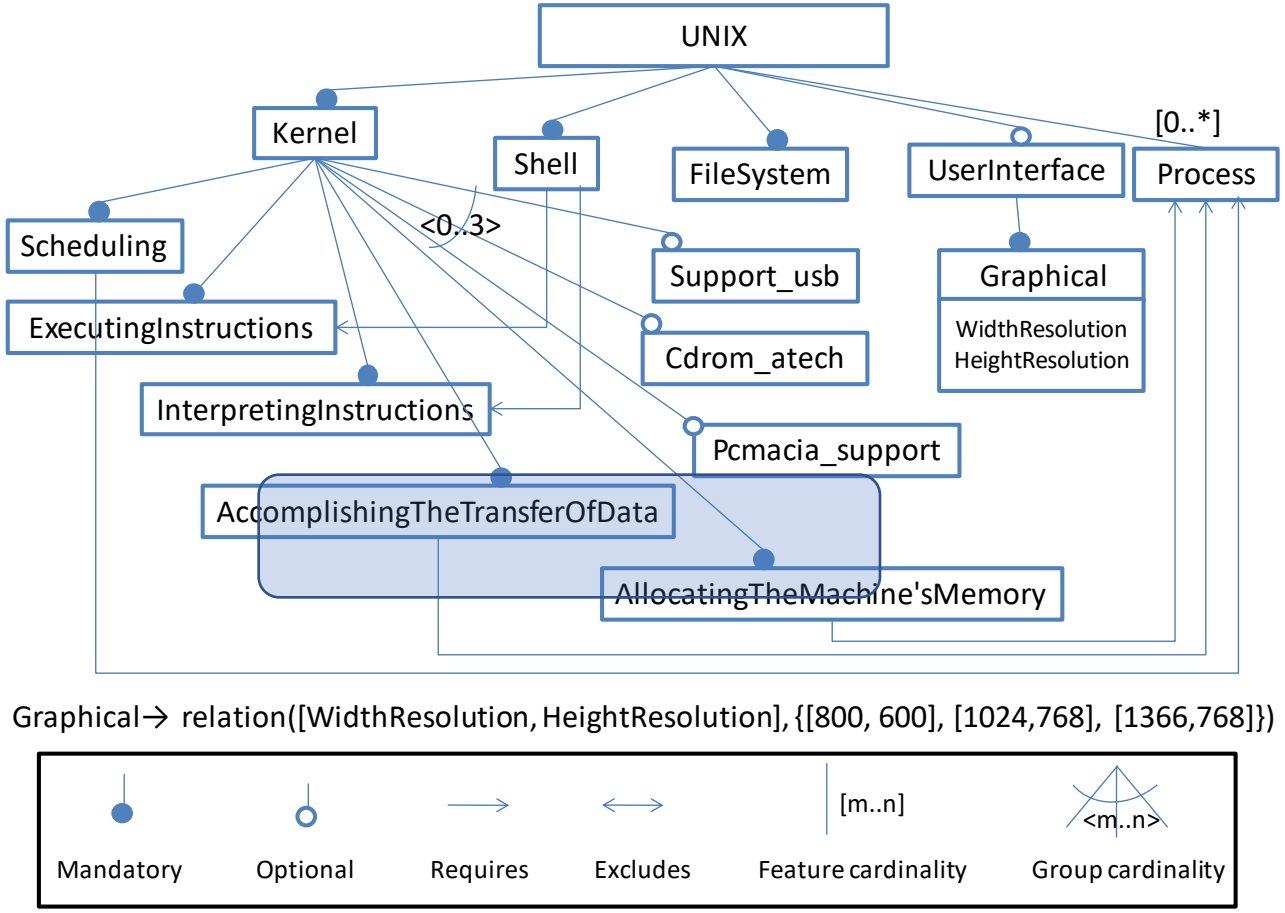
FR_P8.R5	**Configure a text for comparison with another element**
Version	0.7 (01/01/2020)
Description	*In case the action of comparing text is included, those systems of the automation framework product line that only include the option to enter text shall provide the tester with the ability to configure a text for comparison with another element.*
Function/module	Requirements specification module
Type	FR
Variability	Conditionally mandatory.
Priority	Essential requirement.
Delivery date	01/05/2020
Stability	High
Source	Project leader, Raúl Mazo

Terminology issue

- UNIX has one KERNEL.
- Some of the mandatory functions of the KERNEL are:
- ALLOCATING THE MACHINE'S MEMORY to each PROCESS
- SCHEDULING the PROCESSES
- ACCOMPLISHING ... NSFER OF DATA fro... of the m... other
- UNIX can have several PROCE... ...ies even when the UNIX product is full-configured. For the ...
- UNIX offers a logical ... ...d storing large amounts of information in a way tha...
- The KERNEL is composed of static ... ...or a specific hardware platform and cannot be changed, it is called a s... ...n... dynan... modules so that it can 'adapt' to a platform, it is called a dynamic K... inst... odul... USB, CDRO... H, and PCMCIA_SUPPORT cannot be charged, can be charged in a ...tic way or c... ...rged in a ...way.
- The SHELL is a command interpreter; it takes ea... ...ommand and passes it to the KERNEL to be acted upon.
- The GRAPHICAL interface is characterized by a WIDTH RESOLUTION and a HEIGHT RESOLUTION that can have the following couples of values [800,600], [1024,768] and [1366,768].

What if the document (stakeholder descriptions) doesn't use the « right » keywords ?

And then, we design the domain requirements model…



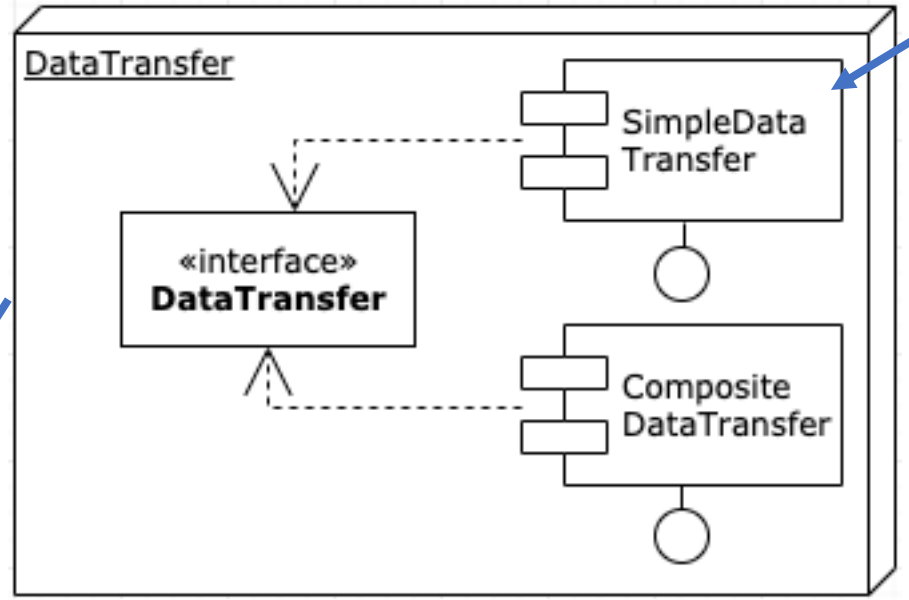
Then, we design, bind and implement the domain reusable components…

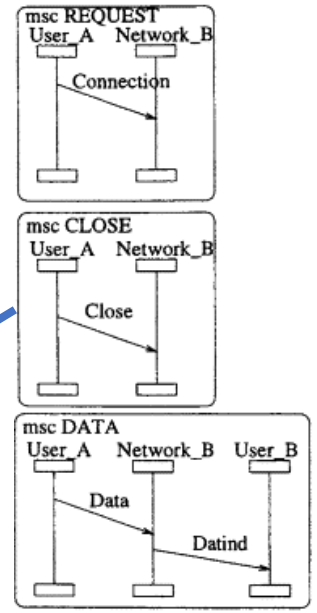
Some of the mandatory functions of the KERNEL are:

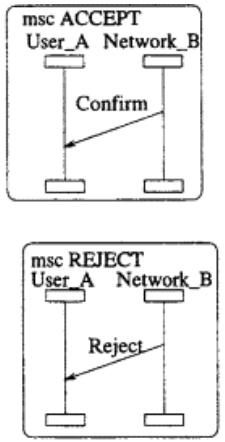
…

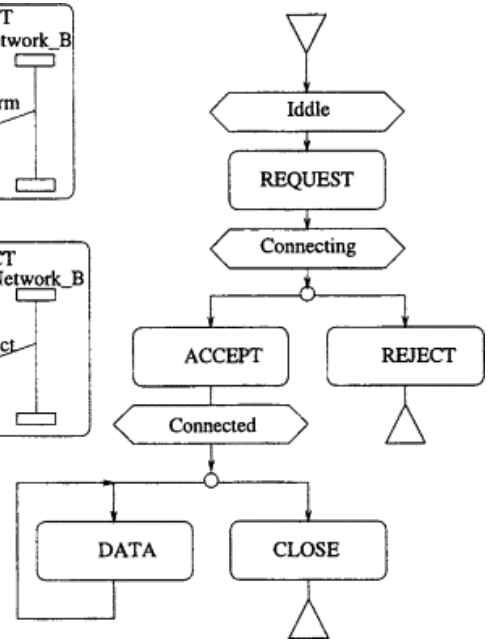
ACCOMPLISHING THE TRANSFER OF DATA from one part of the machine to another

Accomplishing TheTransfer OfData









. . .

Exercise

Create the variability model (e.g., feature model) based on the following requirements and constraint of an appliance store product line.

R1: **SOME** SYSTEMS OF THE appliance store PRODUCT LINE **SHOULD** PROVIDE the clients WITH THE ABILITY TO **manage the products** sold in the stores.

R2: **SOME** SYSTEMS OF THE appliance store PRODUCT LINE **COULD** PROVIDE the clients WITH THE ABILITY TO **validate** themselves in the corresponding system.

R3: **ALL** the SYSTEMS OF THE appliance store PRODUCT LINE **SHALL** PROVIDE the clients WITH THE ABILITY TO **manage the payments** of the products sold in the stores.

R4: **SOME** SYSTEMS OF THE appliance store PRODUCT LINE **COULD** PROVIDE the clients WITH THE ABILITY TO **pay by card**.

R5: **SOME** SYSTEMS OF THE appliance store PRODUCT LINE **COULD** PROVIDE the clients WITH THE ABILITY TO **pay cash**.

C1: The client should be validated into each store system before managing the products of the corresponding store.

Solution to the exercise

Create the variability model (e.g., feature model) based on the following requirements and constraint of an appliance store product line.

R1: **SOME** SYSTEMS OF THE appliance store PRODUCT LINE **SHOULD** PROVIDE the clients WITH THE ABILITY TO **manage the products** sold in the stores.

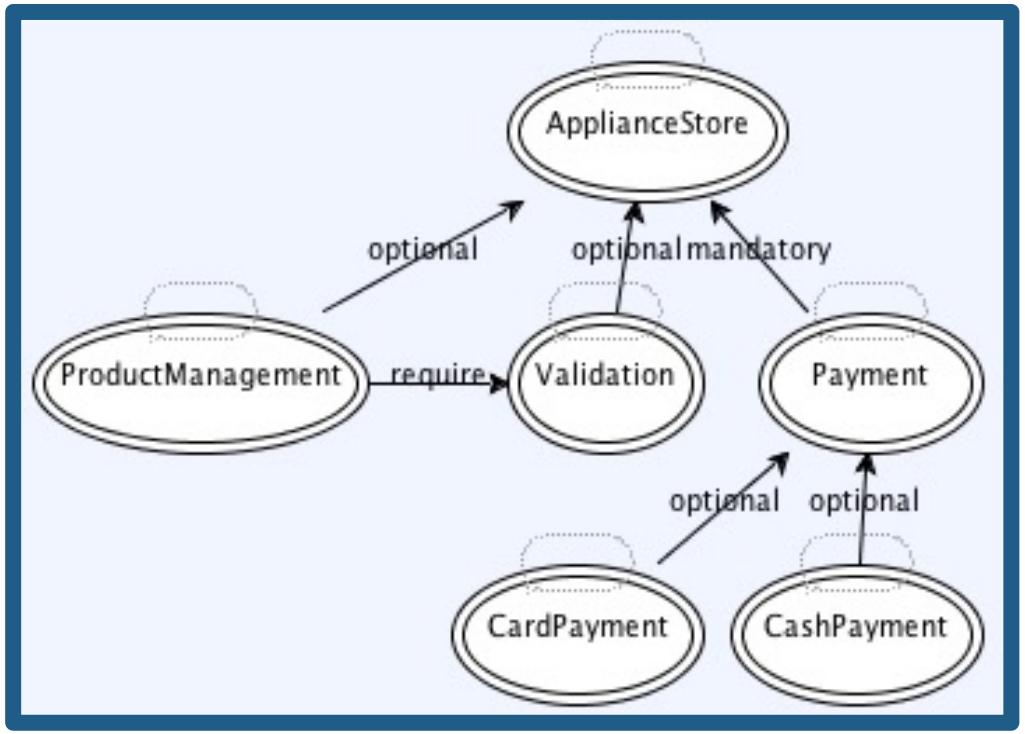
R2: **SOME** SYSTEMS OF THE appliance store PRODUCT LINE **COULD** PROVIDE the clients WITH THE ABILITY TO **validate** themselves in the corresponding system.

R3: **ALL** the SYSTEMS OF THE appliance store PRODUCT LINE **SHALL** PROVIDE the clients WITH THE ABILITY TO **manage the payments** of the products sold in the stores.

R4: **SOME** SYSTEMS OF THE appliance store PRODUCT LINE **COULD** PROVIDE the clients WITH THE ABILITY TO **pay by card**.

R5: **SOME** SYSTEMS OF THE appliance store PRODUCT LINE **COULD** PROVIDE the clients WITH THE ABILITY TO **pay cash**.

C1: The client should be **validated** into each store system **before managing the products** of the corresponding store.





1. **Analysis**
   - A. …
   - B. **Strategies to elicitate the requirements of a PL**
     - a. Requirements-driven strategy
     - **b. Low-end vs. high-end product strategy (reference-product strategy)**
     - c. Multiviewpoint strategy

Low-end vs. high-end **product strategy** (or even, reference-product strategy)

1. Select a low-end and a high-end product
   - Compare the properties (material, dimensions, …) capacities (functions) and the qualities (performance, security,…) of these products and complete these characteristics with all the possibilities in the middle.
2. Specify the resulting variations points and the corresponding variants and define if they corresponds to the scope of the product line.
   - All that is in the scope will be part of the requirements of the product line
3. Complete with the “convenient” characteristics according to the objectives, marketing strategy of the product line





Low-end vs. high-end product strategy (or even, **reference-product strategy**)



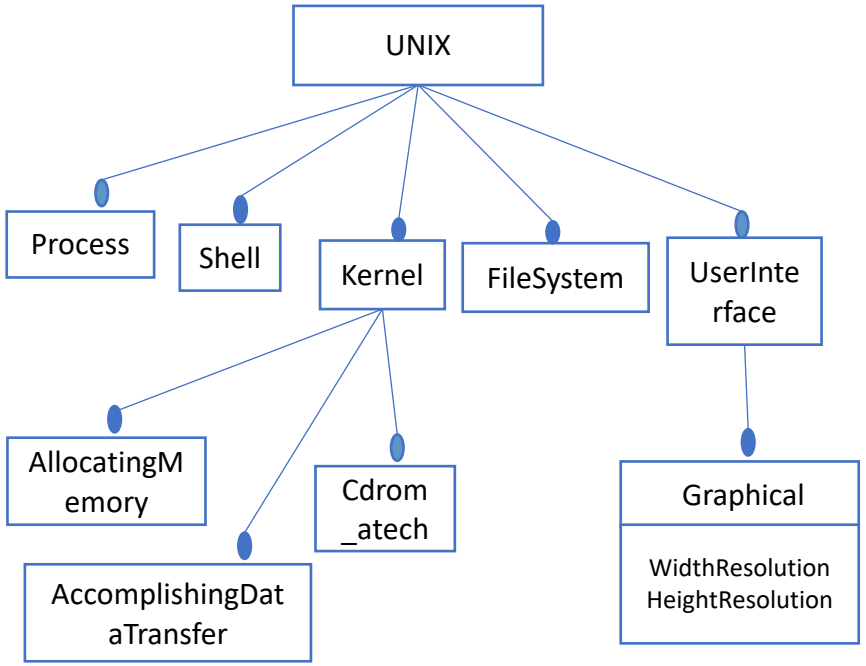
1. **Select reference product**
   - Create breakdown structure => tree
2. Select 2nd product (variant)
   - Identify & integrate variations
   - Identify & integrate constraints
3. Select 3rd product …

Low-end vs. high-end product strategy (or even, **reference-product strategy**)

1. **Select reference product**
   - **Create breakdown structure => tree**
2. Select 2nd product (variant)
   - Identify & integrate variations
   - Identify & integrate constraints
3. Select 3rd product …

redhat

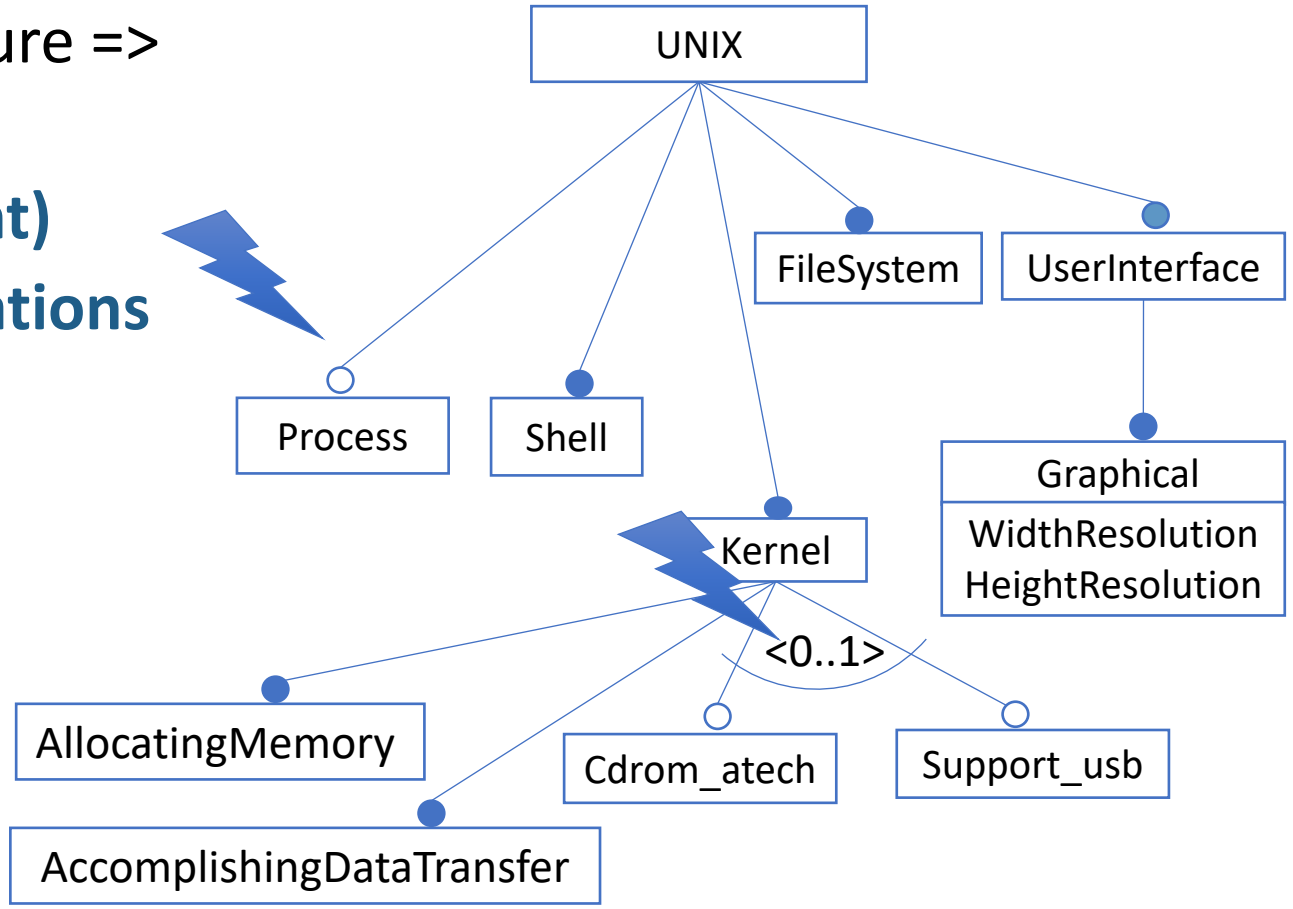
Reference product breakdown structure



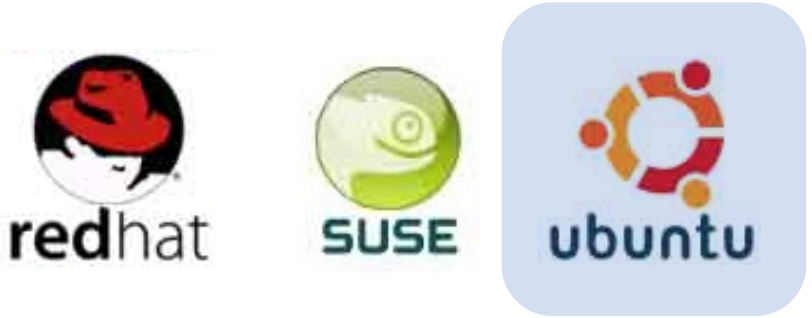
Low-end vs. high-end product strategy (or even, **reference-product strategy**)



1. Select reference product
   - Create breakdown structure => tree
2. **Select 2nd product (variant)**
   - **Identify & integrate variations**
   - **Identify & integrate constraints**
3. Select 3rd product …



Low-end vs. high-end product strategy (or even, **reference-product strategy**)

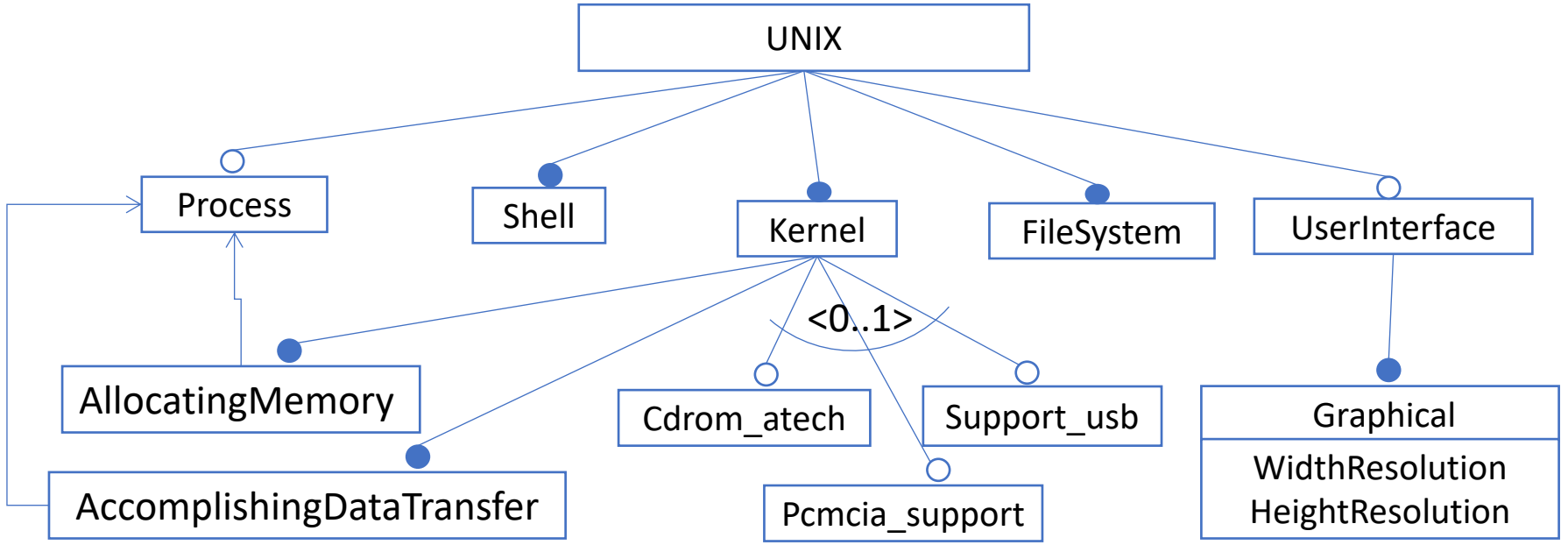


1. Select reference product
   - Create breakdown structure => tree
2. Select 2nd product (variant)
   - Identify & integrate variations
   - Identify & integrate constraints
3. **Select 3rd product …**

And repeat the activities of the second step…
and and so on and so forth!

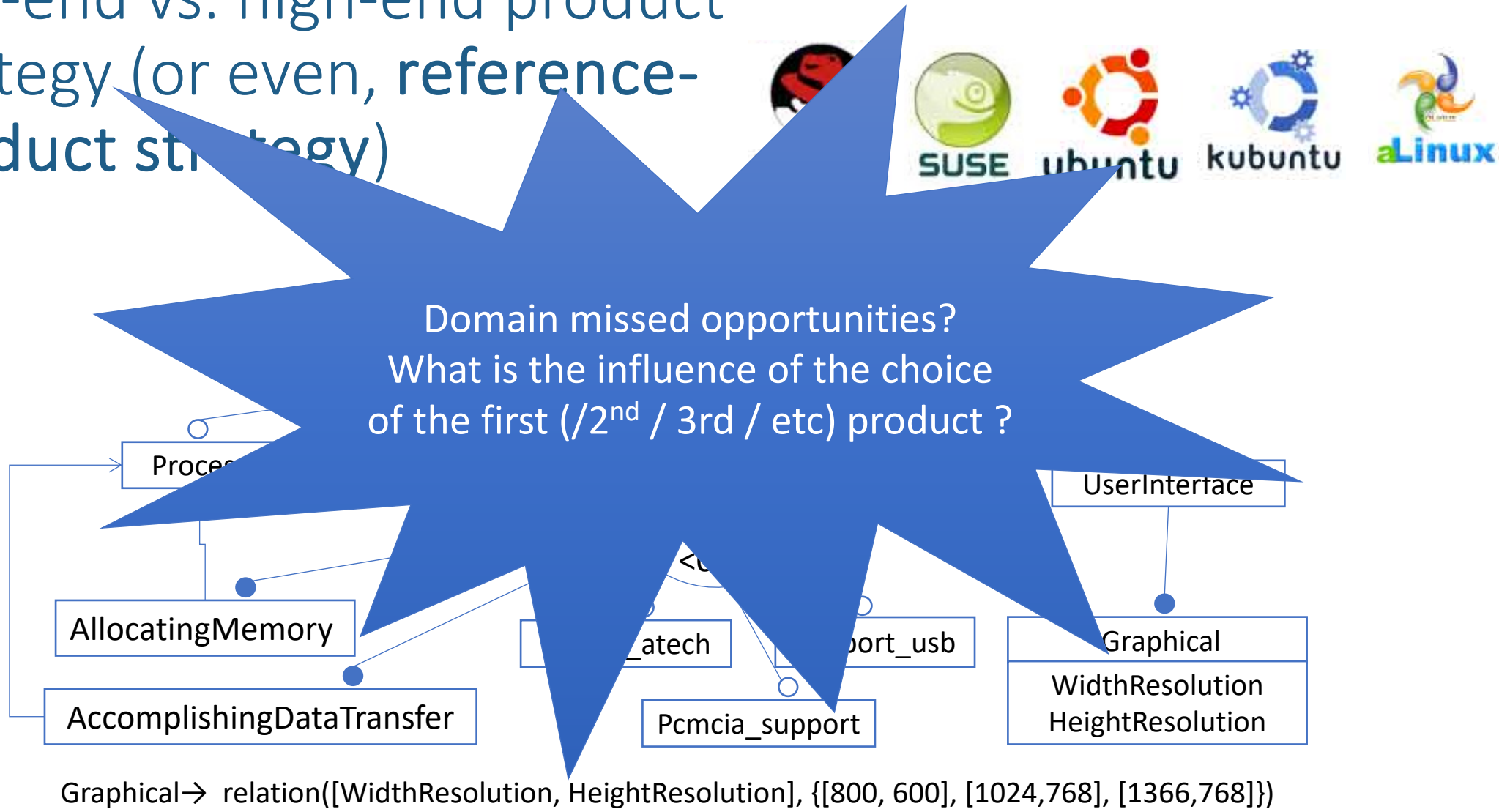
Low-end vs. high-end product strategy (or even, **reference-product strategy**)





Graphical→ relation([WidthResolution, HeightResolution], {[800, 600], [1024,768], [1366,768]})

Low-end vs. high-end product strategy (or even, **reference-product strategy**)





1. Analysis

A. …

B. **Strategies to elicitate the requirements of a PL**

a. Requirements-driven strategy

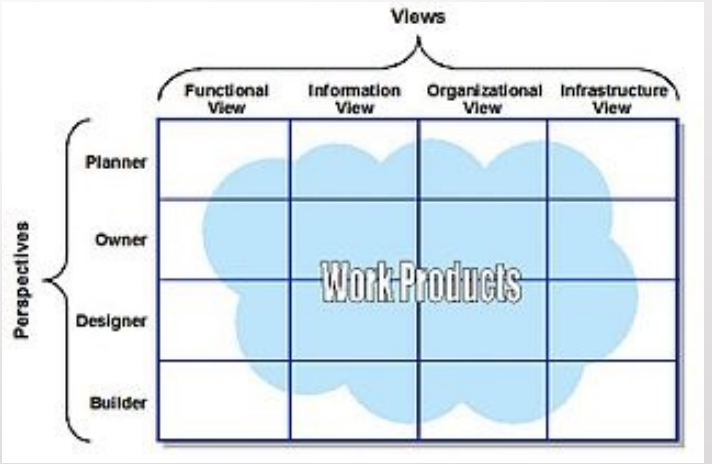
b. Low-end vs. high-end product strategy (reference-product strategy)

c. Multiviewpoint strategy

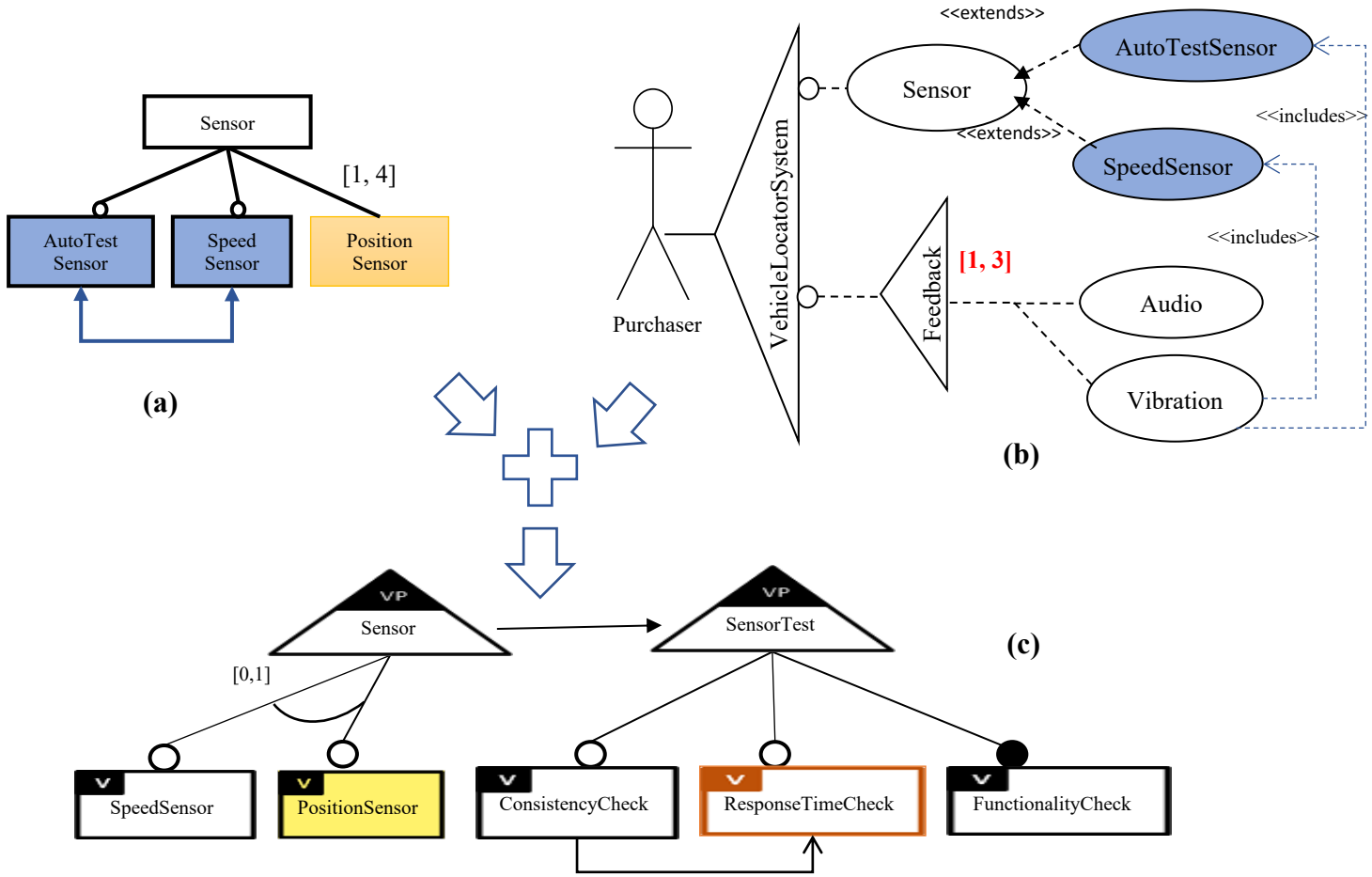
Multi-viepoints strategy

1. Identify viewpoints
2. Specify viewpoints (with/without variability)
3. Identify dependencies
4. Integrate

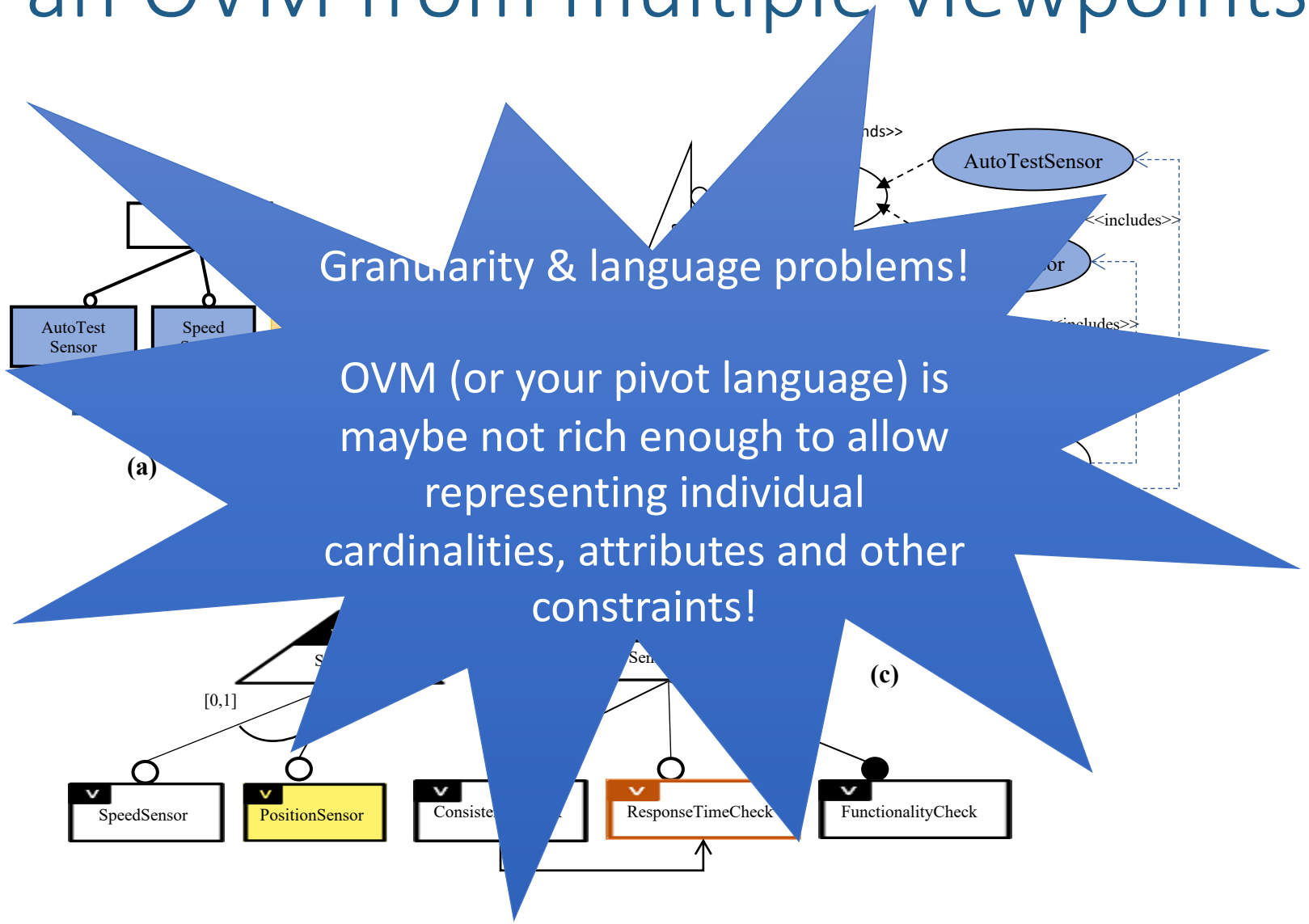




Producing an OVM from multiple viewpoints



Producing an OVM from multiple viewpoints



Integration using OVM as a **pivot** language

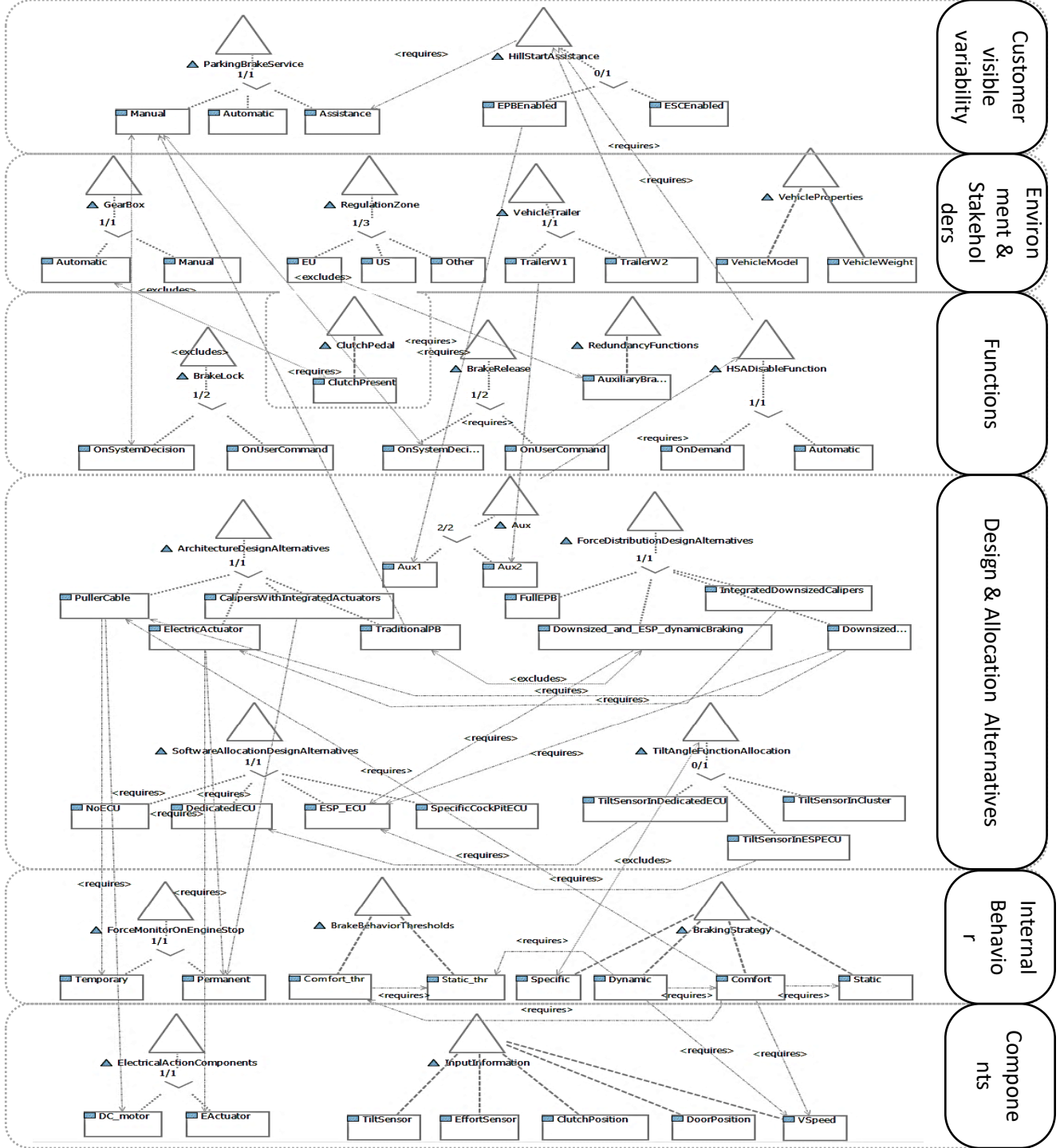
Customer visible variability corresponds to the variability stemming from the vehicle level by the product division.

The context viewpoint contains different facets of the system context, such as: system boundary variability, enabling systems and vehicle environment.

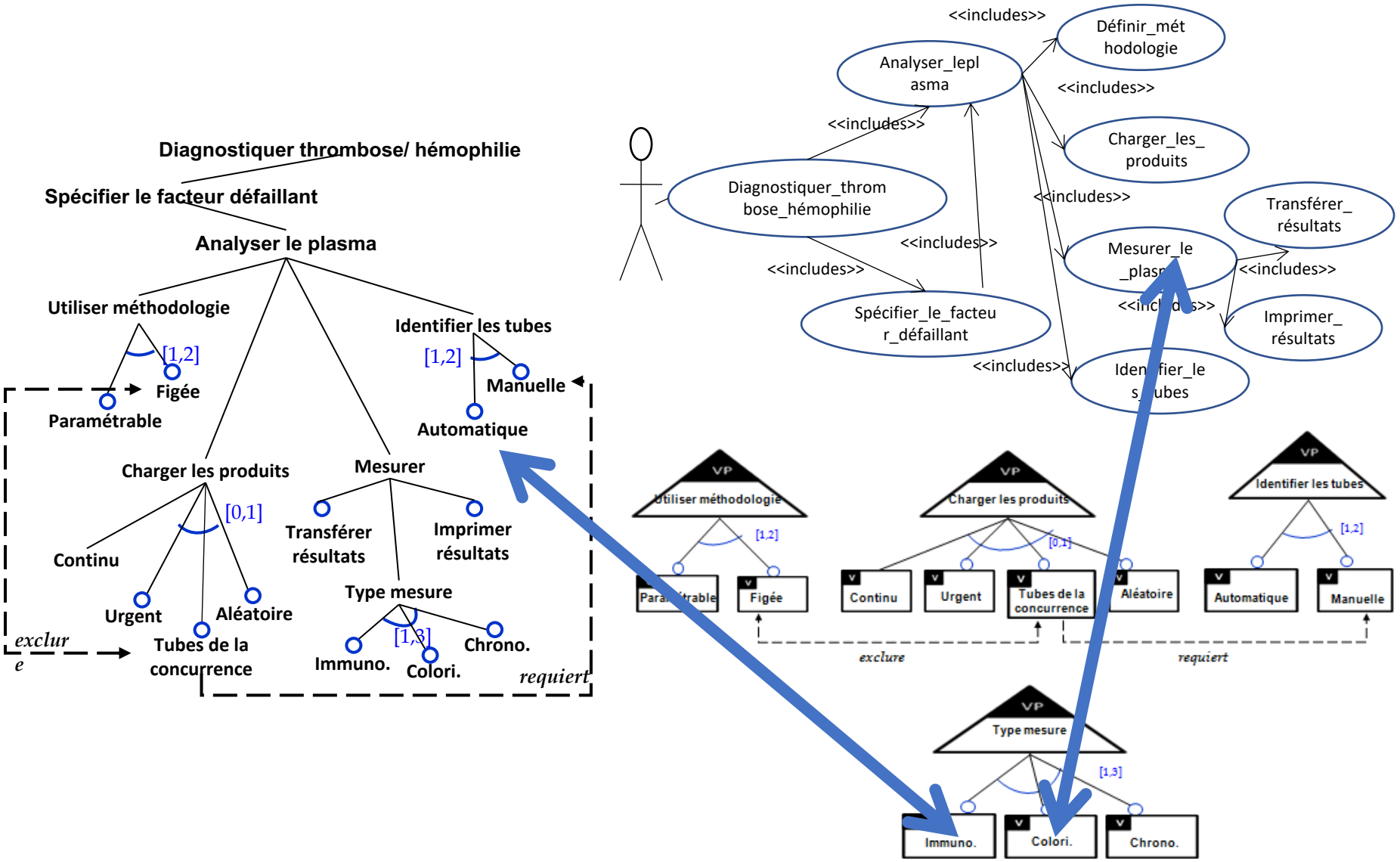
The design alternatives viewpoint specifies design decisions that impact the whole or parts of the technical solution.

The variability entailed by the system internal behavior viewpoint impacts the states and transitions of the system physical and software components.

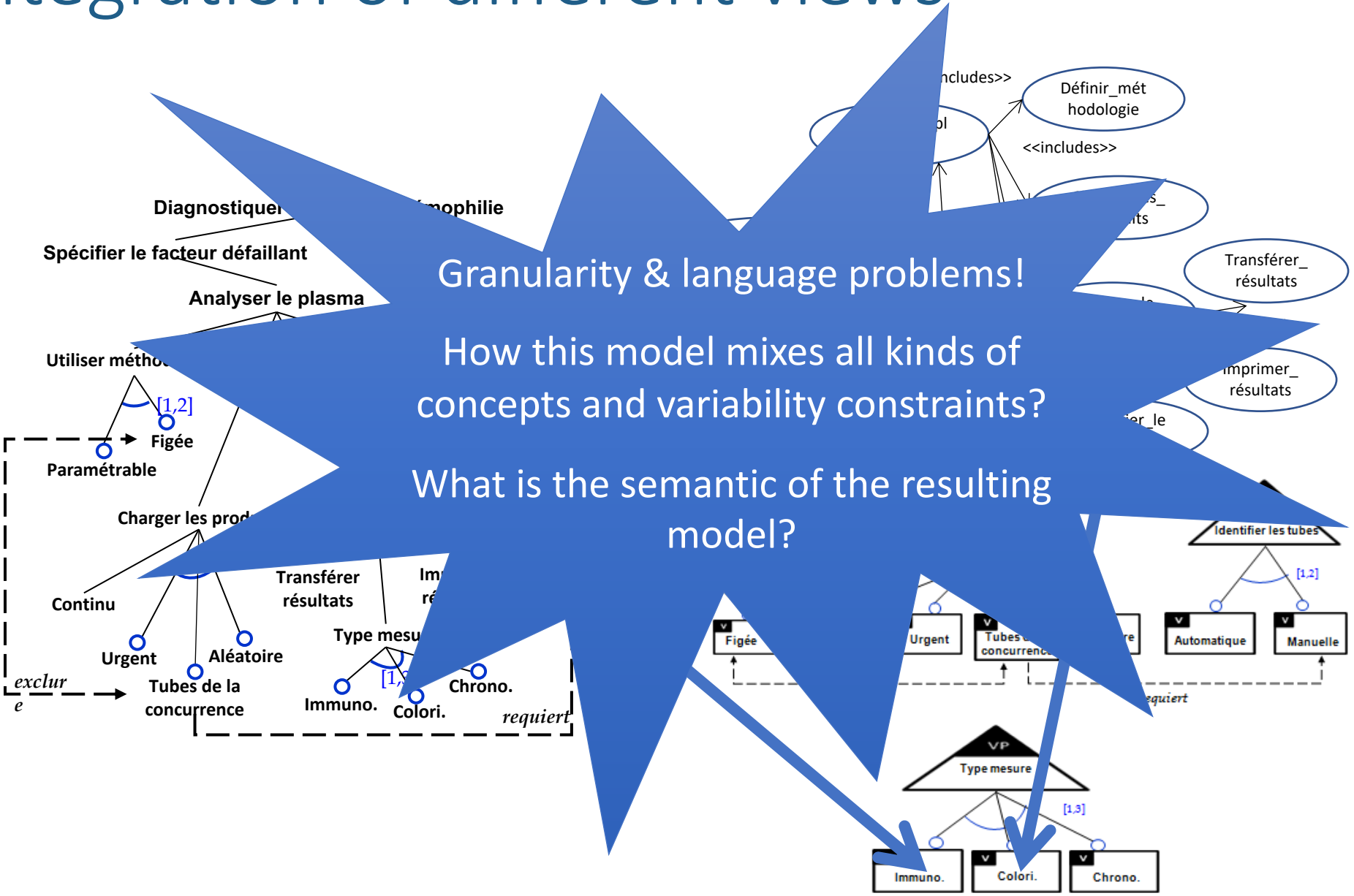
The variability entailed by the physical architecture specifies variability in component decomposition, through optional or replaceable components, as well as physical interfaces variability between components.



Integration of different views



Integration of different views



References and recommended readings

Czarnecki, Krzysztof, Paul Grünbacher, Rick Rabiser, Klaus Schmid, and Andrzej Wąsowski. "Cool features and tough decisions: a comparison of variability modeling approaches." In *Proceedings of the sixth international workshop on variability modeling of software-intensive systems*, pp. 173-182. ACM, 2012.

Cordy, Maxime, Pierre-Yves Schobbens, Patrick Heymans, and Axel Legay. "Beyond boolean product-line model checking: dealing with feature attributes and multi-features." In *Proceedings of the 2013 International Conference on Software Engineering*, pp. 472-481. IEEE Press, 2013.

Dhungana, Deepak, Paul Grünbacher, and Rick Rabiser. "The DOPLER meta-tool for decision-oriented variability modeling: a multiple case study." *Automated Software Engineering* 18, no. 1 (2011): 77-114.

Dumitrescu Cosmin, Dauron Alain, Salinesi Camille, Mazo Raúl. La Réutilisation en Ingénierie de Systèmes à Base de Modèles: Processus et Activités IS pour l'Adoption d'une Approche Lignes de Produits. Genie Logiciel N° 105, pp. 52-59, Paris-France, Juin 2013.

Nair, D.K., 2013. *Variability-Modelling Practices in Industrial Software Product Lines: A Qualitative Study* (Master's thesis, University of Waterloo).

Bühne, Stan, Kim Lauenroth, and Klaus Pohl. "Why is it not sufficient to model requirements variability with feature models." In *Proceedings of Workshop: Automotive Requirements Engineering*, pp. 5-12. 2004.

Mazo Raúl. Avantages et limites des modèles de caractéristiques dans la modélisation des exigences de variabilité. Journal "Génie Logiciel", No. 111, Paris-France, pp. 42-48, Dec. 2014.

Mazo Raúl, Salinesi Camille, Diaz Daniel, Djebbi Olfa, Lora-Michiels Alberto. Constraints: the Heart of Domain and Application Engineering in the Product Lines Engineering Strategy. International Journal of Information System Modeling and Design IJISMD. pp. 33-68. ISSN 1947-8186, eISSN 1947-819, Vol. 3, No. 2. April-June 2012

Dumitrescu Cosmin, Tessier Patrick, Salinesi Camille, Gérard Sebastien, Dauron Alain, Mazo Raúl. Capturing Variability in Model-based Systems Engineering Process. Complex Systems Design & Management (CSD&M) Conference, Springer Press, Paris-France, p. 125-140, 2013.

Birk, A., Heller, G., John, I., Schmid, K., von der Maßen, T. and Muller, K., 2003. Product line engineering, the state of the practice. *IEEE software*, *20*(6), pp.52-60.

Galster, M., Weyns, D., Tofan, D., Michalik, B. and Avgeriou, P., 2014. Variability in software systems—a systematic literature review. *IEEE Transactions on Software Engineering*, *40*(3), pp.282-306.

Mazo Raúl. Guía para la adopción industrial de líneas de productos de software. Editorial Eafit, ISBN 978-958-720-506-0, Medellín-Colombia, 2018.